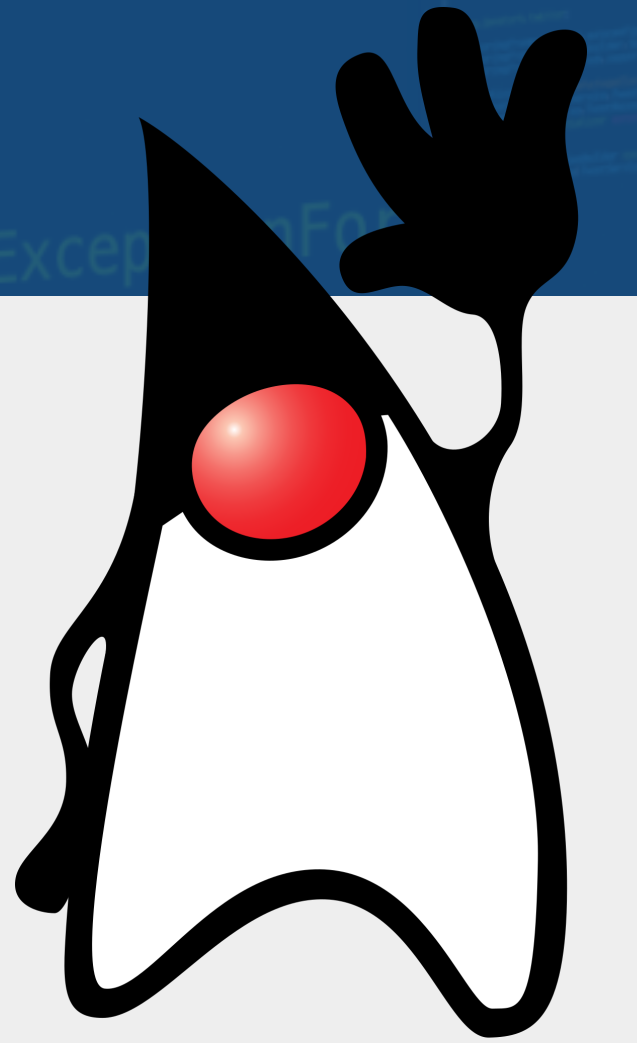


# Java Programlama Dili

## 2. Bölüm

## Java'ya Giriş

Akın Kaldıroğlu





- Bu bölümde Java'yı tanıyacağız, kısa tarihini ve örnek kodlarla en temel özelliklerini öğreneceğiz

# Konular

- **Java'nın Kısa Tarihi**
- **Java'nın Temel Özellikleri**
  - Derleme ve Yorumlama
  - Java Derlenir mi Yorumlanır mı?
  - Java Nesne Merkezlidir
  - İlk Nesne Merkezli Kod
- Bazı Noktalar
- **Java'yı Öğrenmek**
- **Neden Java?**
- **Ödevler**
- **Bundan Sonra?**



# Java'nın Kısa Tarihi



# Sun Microsystems



- Sun Microsystems, 1982 yılında iki mühendis ve iki iş insanının ortak girişimi olarak kuruldu
- Sun, Solaris işletim sistemi ve Sparc CPU ve makinaları (workstation) ile kurumsal Unix platformlarında başarılı bir markaydı
- Unix, en eski işletim sistemlerindendir (İS) ve uzun süre kurumsal (enterprise) dünyanın en çok kullanılan İŞ'ydi, Linux ve sonrasında cloud sistemlerin yaygınlaşmasıyla kullanımı azaldı



# Green Team



- 1991’de Sun içinde Patrick Naughton, Mike Sheridan ve James Gosling önderliğindeki küçük bir takımla bir AR-GE projesi başlatıldı
- “**Green Team**” (**Yeşil Takım**)  
(sonrasında **FirstTeam** ismini aldı) adlı bu takımın amacı ICT (Information & Communication Technology) dünyasına yönelik AR-GE yapmaktı



# \*7 - Star-Seven



- Bu takım 1992 yazında “\*7”, (**Star Seven**) isminde, dokunmalı ekrana sahip bir kontrol cihazı geliştirdi
- “\*7”, TV, video oynatıcısı ve müzik seti gibi pek çok ev cihazını kontrol edebiliyordu
- James Gosling \*7’ni anlatıyor:
  - <https://www.youtube.com/watch?v=1CsTH9S79qI>



# Oak Programlama Dili

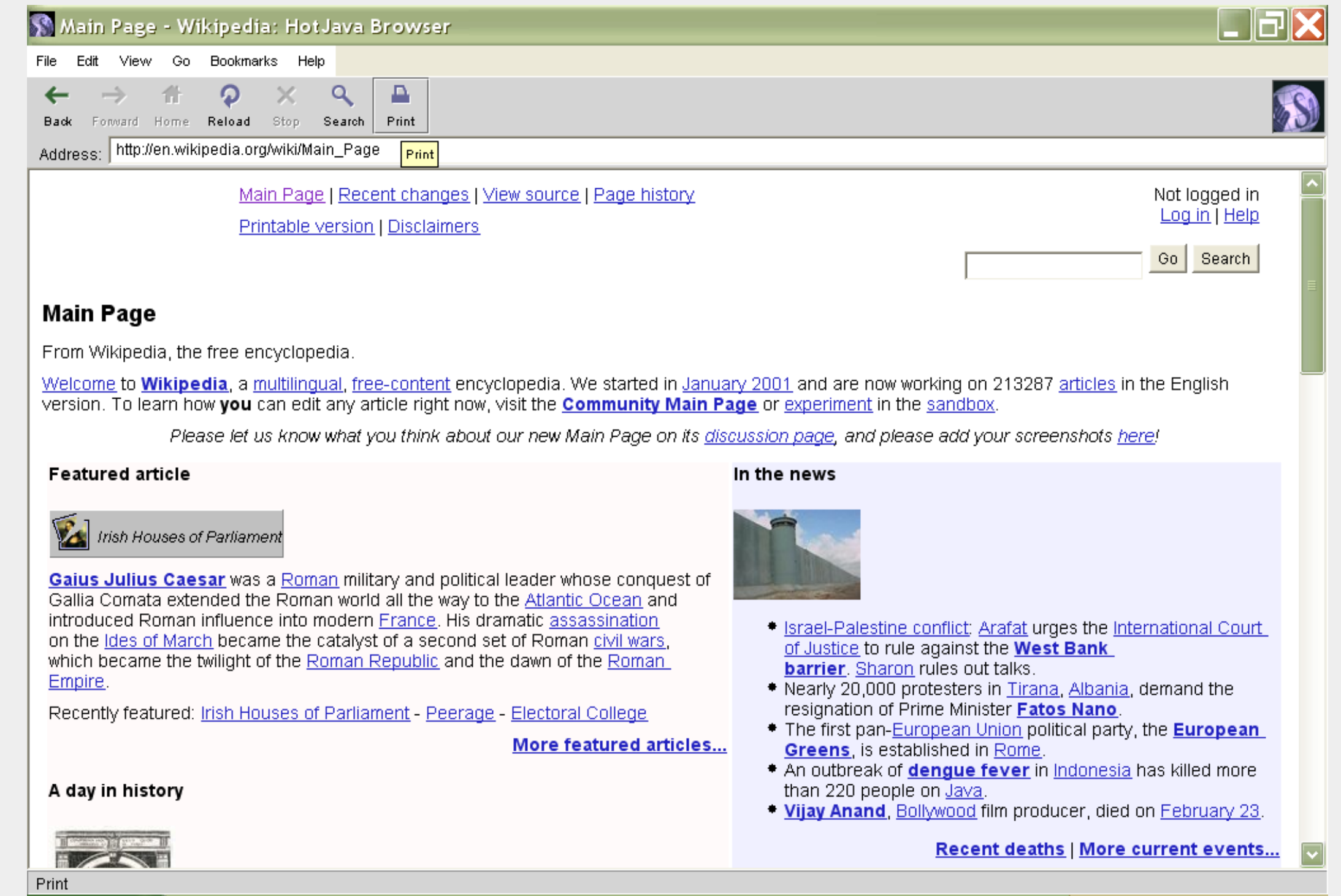


- Gosling, projede önce C++'ı kullanıp, ona bazı eklentiler yapmasına rağmen sonrasında yepyeni bir dil geliştirmenin daha iyi olacağını düşündü
- **Oak** ismini verdiği bu yeni dil, C/C++'a çok benzer söz dizimine sahip, nesne merkezli ve platformdan bağımsızdı
- Gosling ayrıca basit olması için C/C++'ın pek çok özelliğini yeni dilin dışında bırakmıştı

# Yön Değişikliği



- Green Team'in projesi ticari olarak başarılı olamadı
- Ama takımın ileri gelenleri o sırada geliştirdikleri altyapının ve Oak dilinin o günlerde yeni gelişen Internet'e çok uygun olduğunu farkettiler
- Ve hemen yönlerini değiştirip, ismi daha sonra **HotJava** olan Java-tabanlı bir tarayıcı geliştirdiler: **WebRunner**



<https://en.wikipedia.org/wiki/HotJava>

# Gosling'in Hikayesi

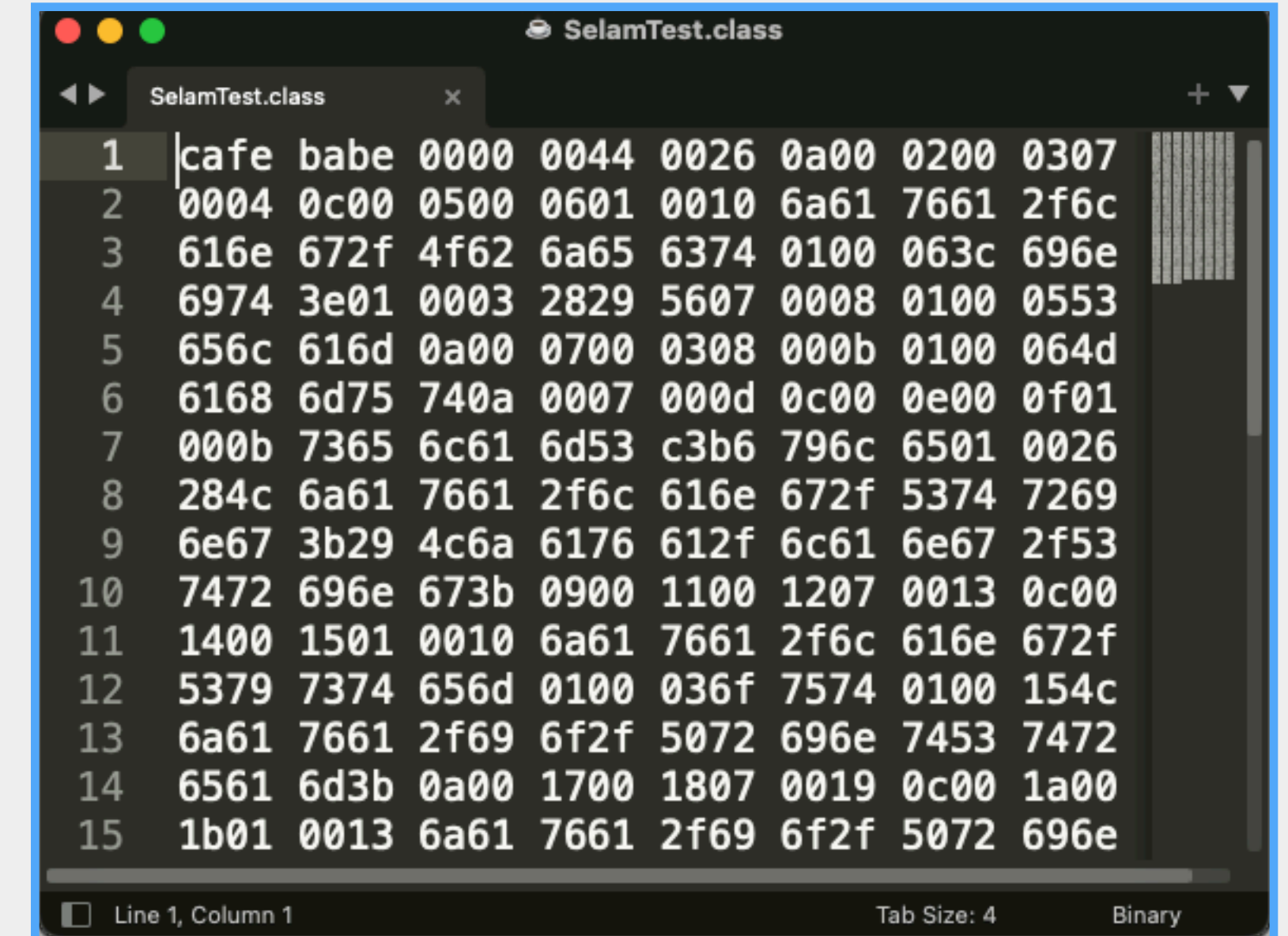


Father of Java, J. Gosling explains: “We had already been developing the kind of *‘underwear’* to make content available at the same time the Web was being developed. Even though the Web had been around for 20 years or so, with FTP and telnet, it was difficult to use. Then Mosaic came out in 1993 as an easy-to-use front end to the Web, and that revolutionized people's perceptions. The Internet was being transformed into exactly the network that we had been trying to convince the cable companies they ought to be building. All the stuff we had wanted to do, in generalities, fit perfectly with the way applications were written, delivered, and used on the Internet. *It was just an incredible accident. And it was patently obvious that the Internet and Java were a match made in heaven. So that's what we did.*”

# Oak'tan Java'ya



- **Oak** dilinin isminin Sun içindeki tartışmalar ve avukatlarla yapılan çalışmalardan sonra **Java** olarak değiştirilmesine karar verildi
- Önceleri Ruby, WRL (Web Runner Language) gibi isimler ortaya atılmış olsa da son oylamaya Java, DNA ve Silk ile girildiğini biliyoruz: <https://www.infoworld.com/article/2170044/so-why-did-they-decide-to-call-it-java-3.html>



# Oak'tan Java'ya



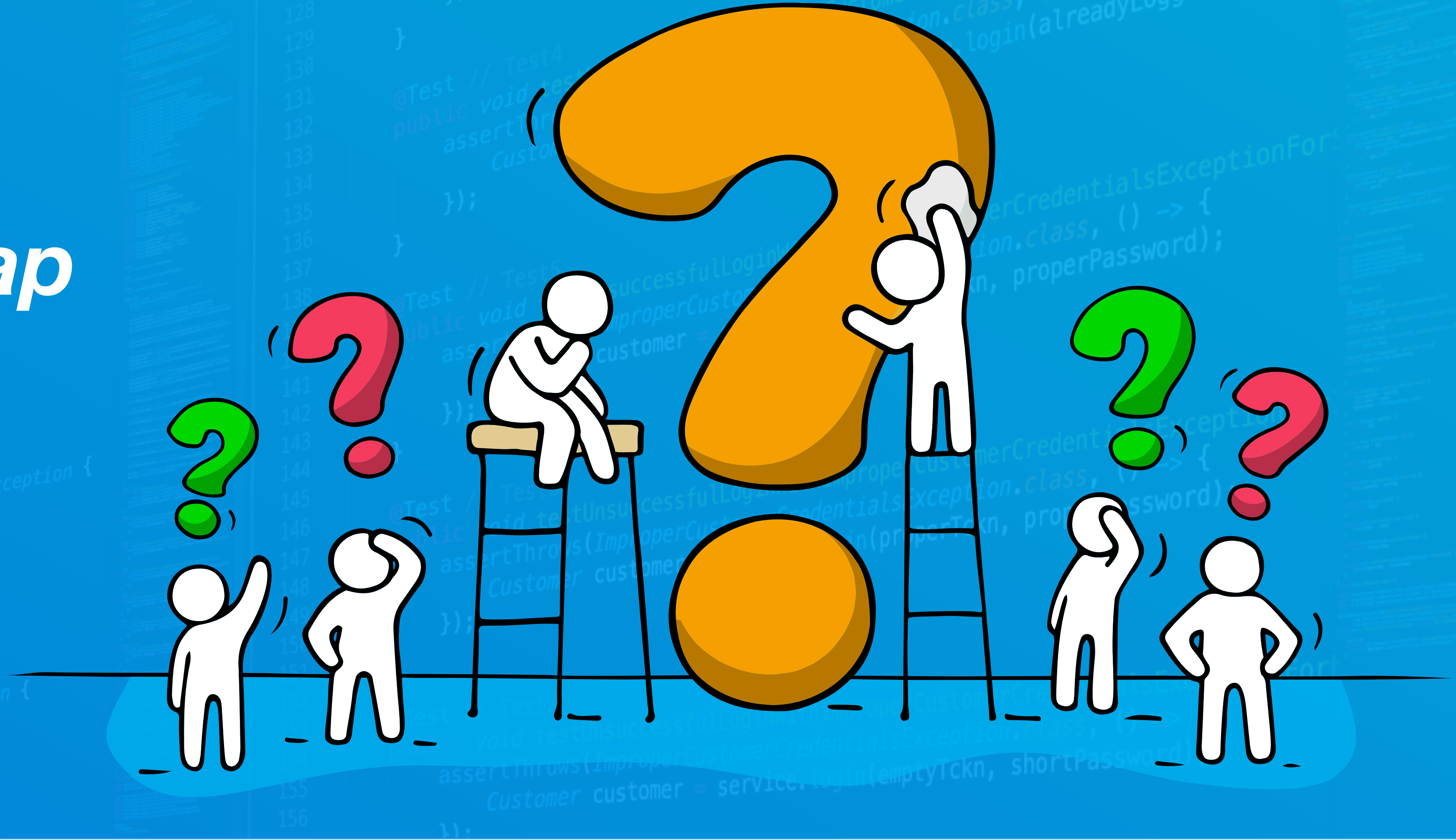
- Java, Sun tarafından **23 Mayıs 1995**'te genel amaçlı bir programlama dili olarak piyasaya sunuldu
- İlk başta pek çokları tarafından “**Web'in dili**” olarak algılandı,
- Netscape'in web yenilikleriyle birlikte gelişti
- Oracle'ın 2010 yılında Sun'ı satın almasıyla Java, Oracle'ın sahipliğinde hayatına devam etmektedir

# 30 Yıl Sonra Hala Java?



- Java, yordamsal (procedural), nesne merkezli (object-oriented) ve fonksiyonel (functional) programlama paradigmalarını bir araya getiren, hem akademide hem endüstride kullanılan bir dil oldu
- Sonrasında frontend - backend ayrışmasıyla backend tarafında en yaygın dil olarak hayatına devam etmektedir

# Soru ve Cevap Zamanı!



# Java'nın Temel Özellikleri



# Java ve C/C++



- Java, söz dizimi (syntax) açısından C++'a çok benzeyen ve temelde Smalltalk ve C++ kültürünü benimseyen bir dildir
- Java, 90lı yıllarda en yaygın sistem geliştirme dili olan C/C++'ın yerini alacak şekilde geliştirildi
- Amaç, Java'nın, C++'dan daha küçük, daha soyut yani daha yukarı seviyede ama daha yetenekli ve güvenilir bir dil olmasıydı
- Bu amaçla C++'ın kullanım zorluğu çıkaran özellikleri dile dahil edilmezken nesne merkezli programlama yetkinlikleri güçlendirildi

# Java Nedir?



Sun, 1995'te Java'yı sunarken yayınladığı ve *Java'nın babası* James Gosling'in kaleme aldığı **The Java Language - A White Paper** başlıklı yazıda Java'yı şöyle tanıtıyordu:

**Java: A simple, object-oriented, network-savvy, interpreted, robust, secure, architecture neutral, portable, high-performance, multithreaded, dynamic language**

**Java: Basit, nesne merkezli, ağlarda yetenekli, yorumlanan, sağlam, güvenli, mimari olarak tarafsız, taşınabilir, yüksek performanslı/başarımlı, çok kanallı, dinamik bir dil**

# Java Basittir



- Java, ataları olan C ve C++'tan daha basittir
- Çünkü onlardaki pek çok karmaşık özelliği dışarıda bırakmıştır:
  - Pointer aritmetiği
  - Programcıya bağımlı bellek yönetimi (memory management)
  - İşlemci çoklu kullanımı (operator overloading)
  - Global değişkenler ve fonksiyonlar
  - Platforma bağımlı yapılar vs.

# Küçük, Soyut ve Güçlü



- Budanan ve yeni gelen özellikler ile Java, C/C++'tan daha küçük ama soyutlama yeteneği açısından daha güçlü bir dil oldu
- Soyut (abstract), nesne merkezli (object-oriented) ve güvenilir (reliable, robust) yapısı, Java'yı özellikle genel amaçlı (general-purpose) kullanım ve iş uygulamaları (business applications) için iyi bir seçenek yaptı
- Ama açıkçası son yıllarda eklenen yeni özelliklerle Java bayağı geniş, C# gibi özellik zengini olmasa bile yine de geniş bir dil haline geldi

# Diğer Özellikleri



- Nesne merkezli (ya da nesne yönelimli) (object-oriented), ağlarda yetenekli (network savvy), sağlam (robust), güvenli (secure) ve çok kanallı (multi-threaded) gibi özellikleri, 90'lı yılların şartlarında ve C ve C++ dillerine göre yorumlamak daha anlamlıdır
- Günümüzde bir dilin bu özelliklere sahip olması sıradan bir durumdur



- Java'nın **dinamik** olmasıyla kastedilen, tipleri çalışma zamanında (run-time) yükleyebilmesidir
- Dolayısıyla Java'da bir programın kullanacağı tipler, uygulamanın derlenmesi ya da ayağa kalkması sırasında hazır olması gerekmez
- Bu durum webin ve dağıtık mimarinin (distributed architecture) doğasına daha uygundur
- Tip açısından Java, **dinamik tipli (dynamically-typed)** değildir, aksine **statik tipli (statically-typed)** bir dildir ve bu konu ileride ele alınacaktır

# Mimari Olarak Tarafsız



- **Mimari olarak tarafsız (architectural-neutral)** olmak, herhangi bir CPU mimarisine bağımlı olmamak demektir
- **Java, standartlar üzerine kurulmuştur,**
  - Örneğin tip büyüklükleri standarttır; `int` her platformda 32-bittir
- Java ve JVM, platformlarla alakalı sadece en genel kabullere sahiptir yani platformdan bağımsızdır (platform independent)

# Taşınabilir



- Java'nın platformlardan bağımsız olması ve yorumlanan (interpreted) tabiatı, onu aynı zamanda **taşınabilir (portable)** kılmaktadır
- Derlenmiş (compiled) Java kodu, `.class` dosyasında bytecode olarak saklanır
- Bytecode platformdan, mimariden bağımsızdır dolayısıyla JVM'in olduğu her ortamda çalıştırılabilir

# WORA ve WOTA



- En başlarda **Bir Kere Yaz Her Yerde Çalıştır (Write Once, Run Anywhere, WORA)** Java'nın taşınabilirlik konusundaki hedefiydi
- Tabi olarak bu prensibin geçerli olması için Java programcısı da platforma özel kod yazmamalıdır
- Ama pratikte geçerli olan, **Bir Kere Yaz Her Yerde Test Et 😊 (Write Once, Test Anywhere, WOTA)** prensibidir

# Yüksek Başarımli



- **Yüksek başarımli (high-performance)** ile aslen, tüm bu özelliklerine rağmen performans açısından iyi bir seviyede olduğu/olabileceği vurgulanmıştır
- Aslen Java'nın performansı ilk yıllarında iyi değildi ama özellikle 2. sürümünden itibaren HotSpot JVM ile performansı iyileşti
- Hemen her sürümde JVM'e performans odaklı yenilikler gelmektedir
- Her halükarda Java geliştiricilerinin performans konusunda bilgili ve dikkatli olmaları gereklidir



# Ödev

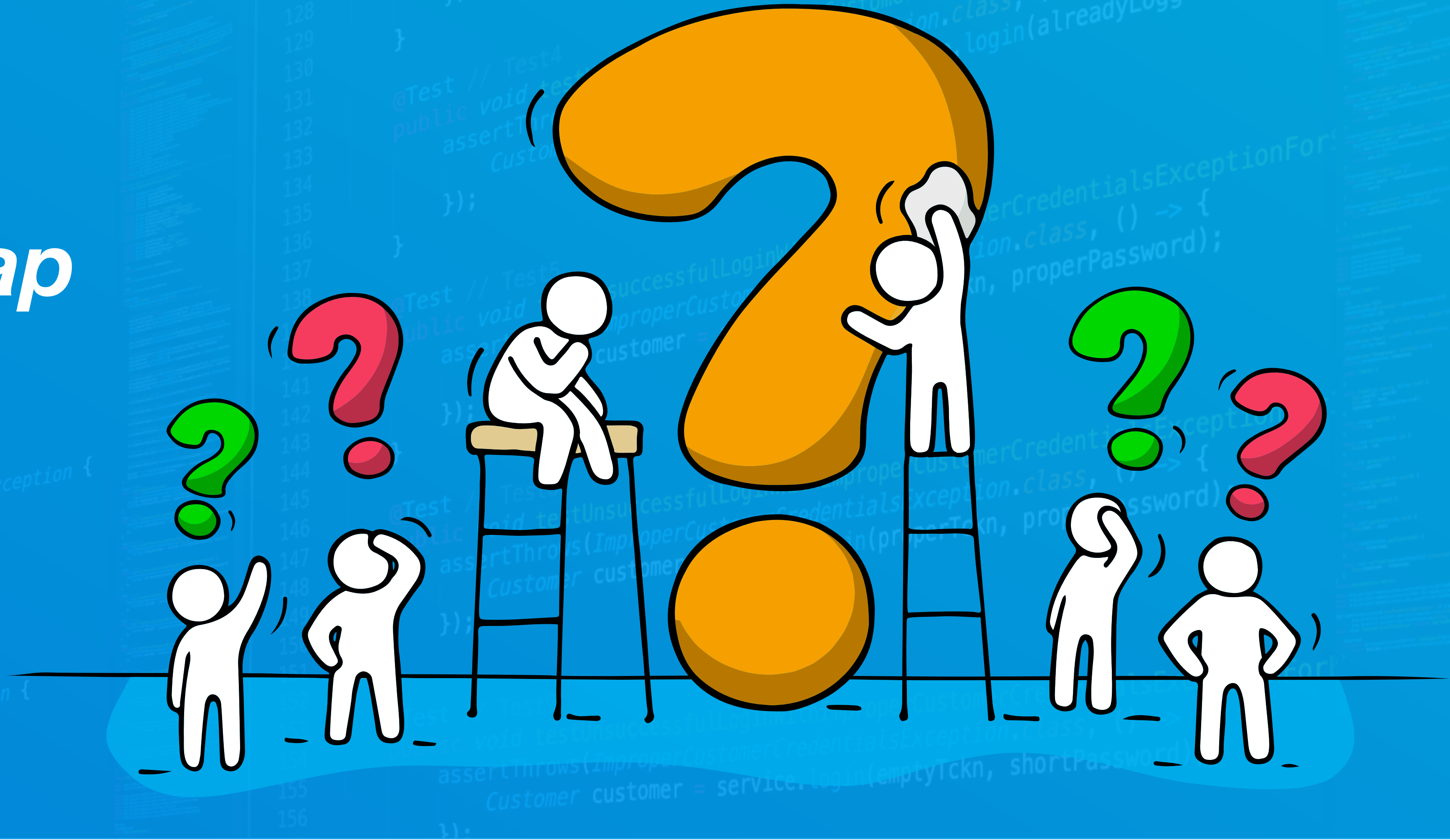


- Java ilk çıktığında yayınlanan bazı makalelere göz atabilirsiniz:
- The Java Language: A White Paper:
  - <https://www.tech-insider.org/java/research/acrobat/9503.pdf>
  - <https://www.tech-insider.org/java/research/acrobat/9503.pdf> (This version includes an additional glossary)
- The Java Language Environment: <https://www.oracle.com/java/technologies/language-environment.html>



- Lex Fridman'ın Youtube kanalında *James Gosling: Java, JVM, Emacs, and the Early Days of Computing* başlıklı, Java'nın çıkışıyla ilgili konuşmaların da geçtiği sohbeti (podcast) dinleyebilirsiniz:
- [https://www.youtube.com/watch?v=IT\\_Nrr3PNI&t=45s&ab\\_channel=LexFridman](https://www.youtube.com/watch?v=IT_Nrr3PNI&t=45s&ab_channel=LexFridman)

# Soru ve Cevap Zamanı!





# Derleme ve Yorumlama

# Derleme ve Yorumlama



- **Derleme (compilation)** ve **yorumlama (interpretation)**, programlama dilleri dünyasının en temel kavramlarındanındır
- Diller sıklıkla derlenen (compiled) ve yorumlanan (interpreted) olarak ikiye ayrılır
- **Compiled (derlenen)**: Fortran, C, C++, Pascal, Haskell, Rust, Go
- **Interpreted (yorumlanan)**: JavaScript, PHP, Python, Ruby

# Derleme (Compilation) - I



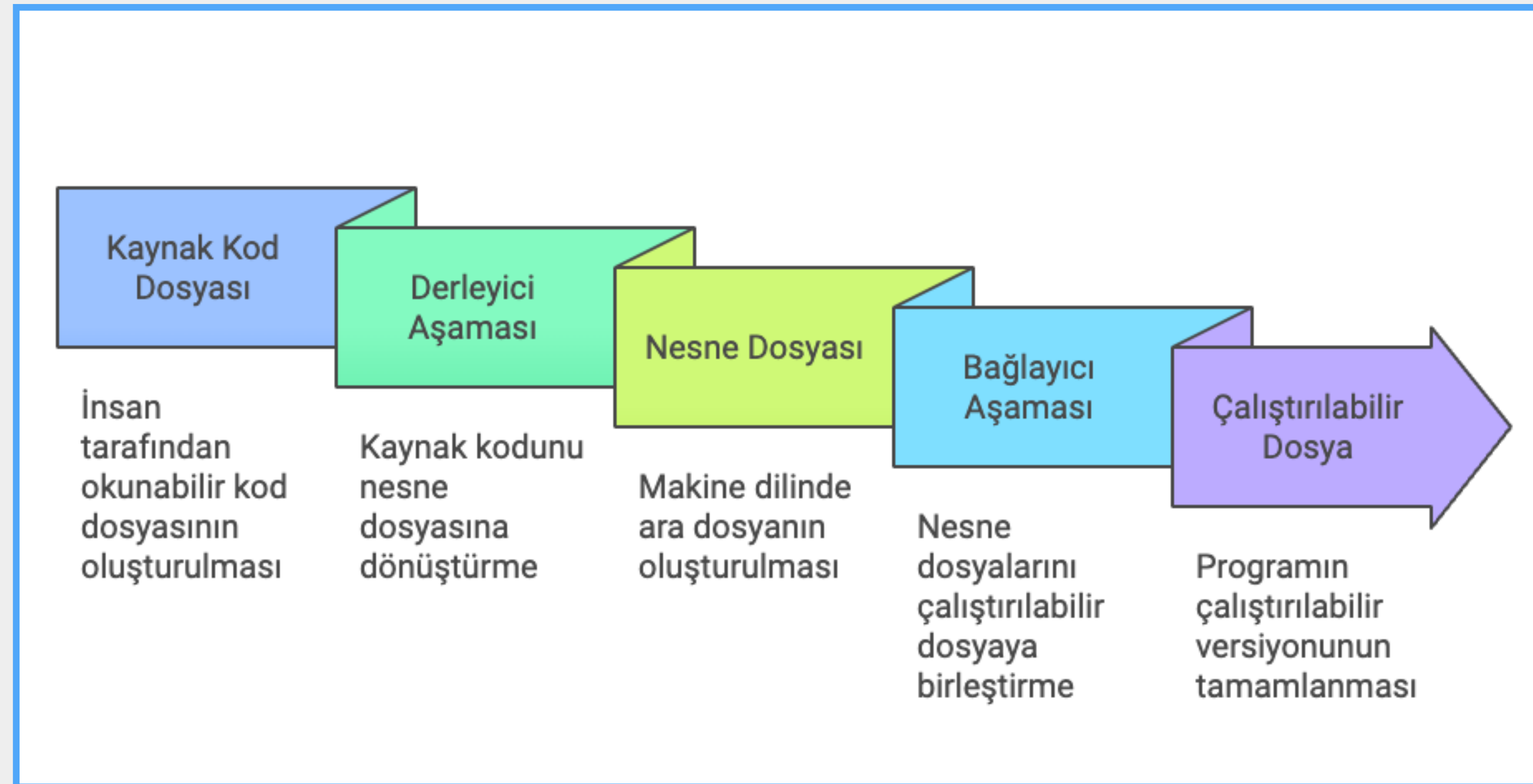
- Genel olarak **derleme (compilation)**, kaynak kodun **derleyici (compiler)** tarafından bir formdan başka bir forma çevrilmesidir; bu sebeple buna çeviri (translation) de denir
- Tipik olarak, derleme, kaynak kodun, **çalıştırılabilir (executable)** makina koduna çevrilmesidir
- Örneğin insanların yazıp okuyabildiği (human-readable) **cpp** uzantılı C++ kaynak kodundan (source code) ikili (binary), insanların okuyamadığı (human-unreadable) **çalıştırılabilir (executable, exe, out)** kod üretmektir

# Derleme (Compilation) - II



- Kaynak kod, önce derleyici tarafından derlenir (compilation), sonra kullandığı kütüphaneler ile ilişkilendirilir (linking) ve makina kodu içeren çalıştırılabilir (executable) kod üretilir
- Sonrasında çalıştırılabilir kod, çalıştırılır (run)

# Derleme (Compilation) - III



napkin.ai

# Derleme ve Çalışma Zamanı



- Dolayısıyla derlenen dillerde kod ile ilgili iki ayrı zaman dilimi söz konusudur:
  - Kaynak kodun derlendiği zaman, **derleme zamanı (compile-time)**
  - Derlenen kodun çalıştırıldığı zaman, **çalışma zamanı (run-time)**
- Yazılımcılar hataların derleme zamanında ortaya çıkmasını umarlarken çalışma zamanının problemsiz ve yüksek performansı olmasını beklerler

# Monte Carlo Simülasyonu ile PI - I



- Şimdi IntelliJ'deki projenizdeki `ch02` isimli dizini, masa üstündeki (desktop) `Code` isimli dizine kopyalayın
- `ch02` içindeki `pi` dizininde PI sayısını Monte Carlo simülasyonu ile bulan C++, Python ve Java kodlarını içeren 3 ayrı dosya göreceksiniz

```
akin@M3-Max:~/Desktop/Code/ch02/pi
/Users/akin/Desktop/Code/ch02 >ls -l
total 0
drwxr-xr-x@ 7 akin  staff  224 20 Ara 08:35 pi
drwxr-xr-x@ 11 akin  staff  352 15 Kas 23:43 selam
/Users/akin/Desktop/Code/ch02 >cd pi
/Users/akin/Desktop/Code/ch02/pi >ls
MonteCarloPI.cpp      MonteCarloPI.py
MonteCarloPI.java     Readme.txt
/Users/akin/Desktop/Code/ch02/pi >
```

# Monte Carlo Simülasyonu ile PI - II



- Şimdi `pi` dizininde PI sayısını Monte Carlo simülasyonu ile bulan daha doğrusu yakınsayan C++, Python ve Java kodlarını çalıştıracamız ve bunu yaparken derleme ve yorumlama süreçlerini ve farklarını göreceğiz
- Herkes kendi platformunda istediği derleyici ve yorumlayıcı kullanarak C++ ve Python kodlarını çalıştırabilir

```
#include <iostream>
#include <stdlib.h> // srand
#include <ctime>

using namespace std;

# define PI_L  3.141592653589793238462643383279502884L

int main() {
    cout.precision(10);
    int n;
    int dotsInCircle = 0;
    cout << "Please enter number of points: " << endl;
    cin >> n;
    clock_t start = clock();
    for (int i = 0; i < n; i++) {
        double x = static_cast<double>(rand()) /
            static_cast<double>(RAND_MAX);
        double y = static_cast<double>(rand()) /
            static_cast<double>(RAND_MAX);
        double distance = x * x + y * y;
        if (distance <= 1.0)
            dotsInCircle++;
    }
    clock_t end = clock();
    float time = ((float) end - start) / CLOCKS_PER_SEC;
    double myPi = (double) 4 * dotsInCircle / n;

    cout << "PI: " << PI_L << endl;
    cout << "Calculated PI: " << myPi << endl;
    cout << "Time is: " << time << " s." << endl;

    return 0;
}
```

# C++ Kodunu Derleme ve Çalıştırma - II



- `pi` dizinine gidin ve `MonteCarloPI.cpp` dosyasındaki C++ kodunu bir C++ derleyicisi ile derleyin
- macOS üzerinde GNU derleyicisiyle, `g++ -O3 -std=c++23 -o MonteCarloPI.exe MonteCarloPI.cpp` komutuyla `MonteCarloPI.cpp` dosyası derlenir ve çalıştırılabilir  
`MonteCarloPI.exe` kodu üretilir
- Sonrasında bu kodu çalıştırın

```
#include <iostream>
#include <stdlib.h> // srand
#include <ctime>

using namespace std;

# define PI_L 3.141592653589793238462643383279502884L

int main() {
    cout.precision(10);
    int n;
    int dotsInCircle = 0;
    cout << "Please enter number of points: " << endl;
    cin >> n;
    clock_t start = clock();
    for (int i = 0; i < n; i++) {
        double x = static_cast<double>(rand()) /
            static_cast<double>(RAND_MAX);
        double y = static_cast<double>(rand()) /
            static_cast<double>(RAND_MAX);
        double distance = x * x + y * y;
        if (distance <= 1.0)
            dotsInCircle++;
    }
    clock_t end = clock();
    float time = ((float) end - start) / CLOCKS_PER_SEC;
    double myPi = (double) 4 * dotsInCircle / n;

    cout << "PI: " << PI_L << endl;
    cout << "Calculated PI: " << myPi << endl;
    cout << "Time is: " << time << " s." << endl;

    return 0;
}
```

# Yorumlama (Interpretation) - I

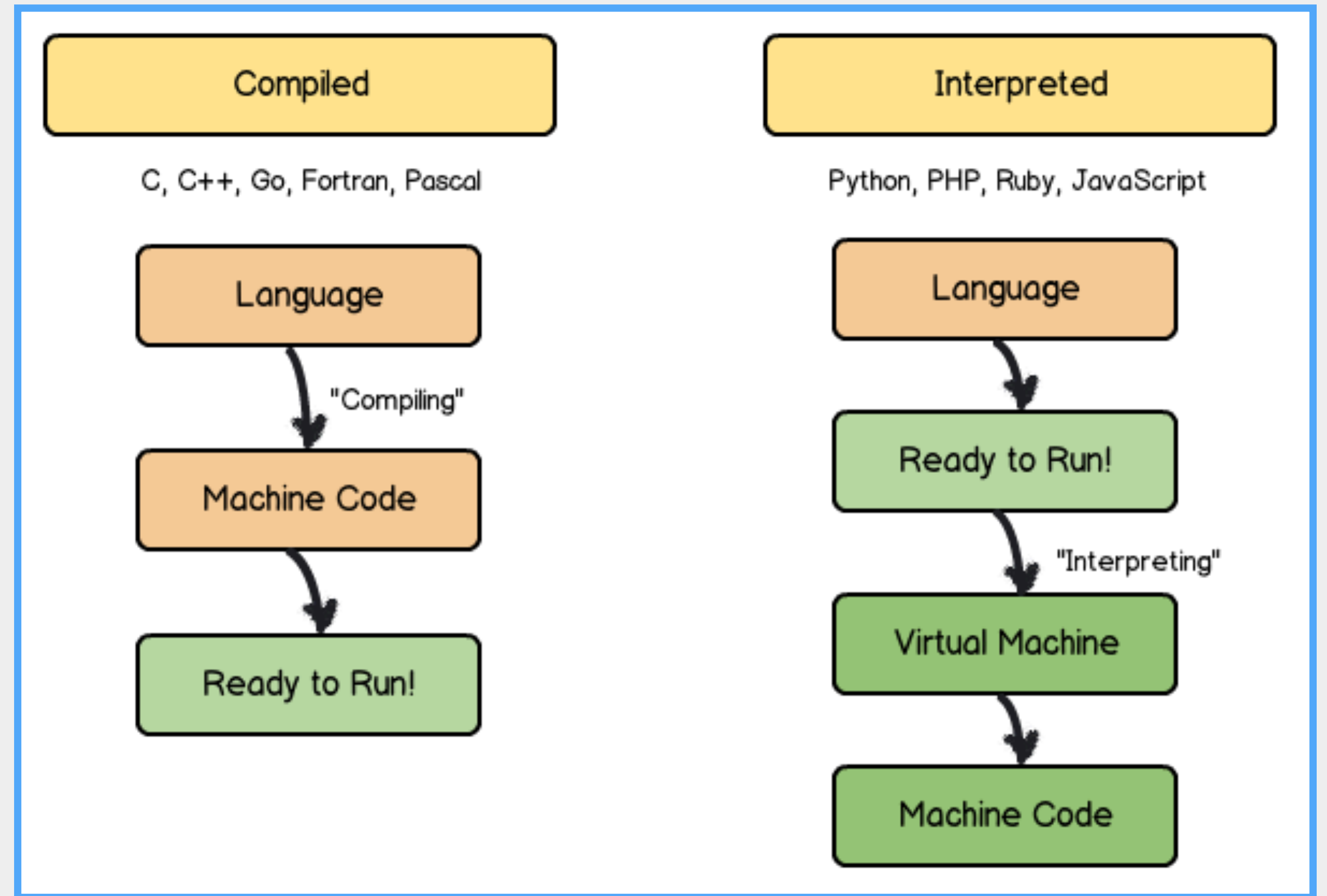


- Yorumlama ile kaynak kod, başka bir forma çevrilmeden, **yorumlayıcı (interpreter)** tarafından, doğrudan makina kodundaki bloklar çağrılarak çalıştırılır yani yorumlanır (interpretation)
- Bilinen, klasik yaklaşımda, kaynak kod satır satır çalıştırılır, yorumlanır
- Derlemede olduğu gibi bir çıktı, son, çalıştırılabilir kodu içeren bir dosya yoktur
- Dolayısıyla yorumlamada derleme zamanı yoktur, sadece çalışma zamanı vardır

# Yorumlama (Interpretation) - II



- Daha modern yapılarda kaynak kod çalıştırılmadan önce, bytecode da denilen bir başka forma çevrilir ve bu formdaki kod sanal bir makina üzerinde yorumlanır
- Her halükarda derlemede olduğu gibi bir çıktı, son, çalıştırılabilir kodu içeren bir dosya yoktur



<https://medium.com/@astermanuelg/blurred-lines-is-ruby-an-interpreted-language-2d3d6bca3d37>

# Python Kodunu Çalıştırma



- Sonra aynı dizinde `MonteCarloPI.py` dosyasındaki Python kodunu, Python yorumlayıcısıyla çalıştırın
- Bunun için ben makinamda `python3 MonteCarloPI.py` komutunu kullanıyorum

```
import random as r
import math as m
import time

# Total number of point to throw.
s = input("Please enter number of points: ")
n = int(s)

dotsInCircle = 0

start = time.clock()
# Iterate for the number of darts.
for i in range(0, n):
    # Generate random x, y in [0, 1].
    x = r.random()
    y = r.random()
    # Increment if inside unit circle.
    if (x*x + y*y) < 1.0:
        dotsInCircle += 1

end = time.clock()

# inside / total = pi / 4
pi = (float(dotsInCircle) / n) * 4

print "PI: ", m.pi
print "Calculated PI: ", pi
seconds_elapsed = (end - start)
print "Time is ", seconds_elapsed, " s."
```

# Derleme ve Yorumlama Farkı - I



- Örneklerden de görüldüğü gibi derlemeyle, insanın okuyabileceği kaynak koddan (source code), ikili (binary) formatta çalıştırılabilir (executable) bir kod üretilir ve ayrı bir dosyada saklanır
- Dolayısıyla kaynak kod çalıştırılmaz, ondan derlenerek üretilen kod çalıştırılır
- Ama yorumlanan dillerde kaynak kod herhangi bir başka forma çevrilmeden doğrudan çalıştırılır
- Bu iki çalışma şeklinin tabi olarak artıları ve eksileri vardır

# Derleme ve Yorumlama Farkı - II



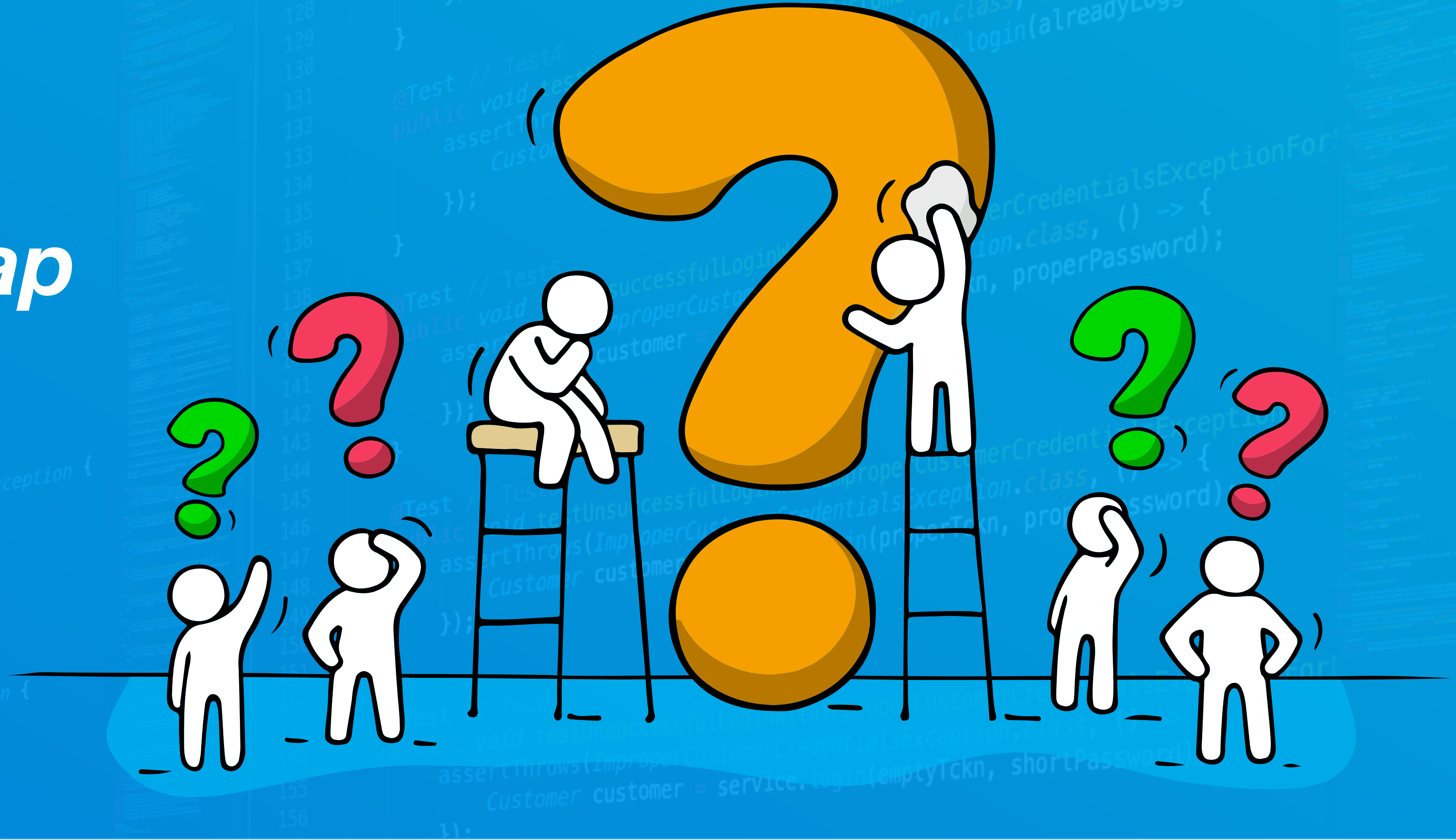
- Derlenen kodlar, çalışma zamanında (run-time) daha etkindirler çünkü hem hatadan temizlenmişlerdir hem de çalışma-zamanına daha az iş bırakmışlardır dolayısıyla performanslıdırlar
- Ama makina koduna derlenmiş kod, platforma bağımlıdır dolayısıyla farklı platformlarda çalıştırmak istenirse aynı kaynak kod her farklı platform için ayrı ayrı derlenmelidir
- Ayrıca kaynak koduna yapılacak herhangi bir değişiklik tekrar derlemeyi gerekli kılar

# Derleme ve Yorumlama Farkı - II



- Yorumlama, kodu daha dinamik ve platformdan bağımsız yapar
- Lakin hatalar çalışma zamanında ortaya çıkar
- Kaynak koda yapılan değişiklikler çalışma zamanında görünür
- Derlemede çalışma zamanı performansı, yorumlamada ise platformdan bağımsızlık öne çıkar

# Soru ve Cevap Zamanı!





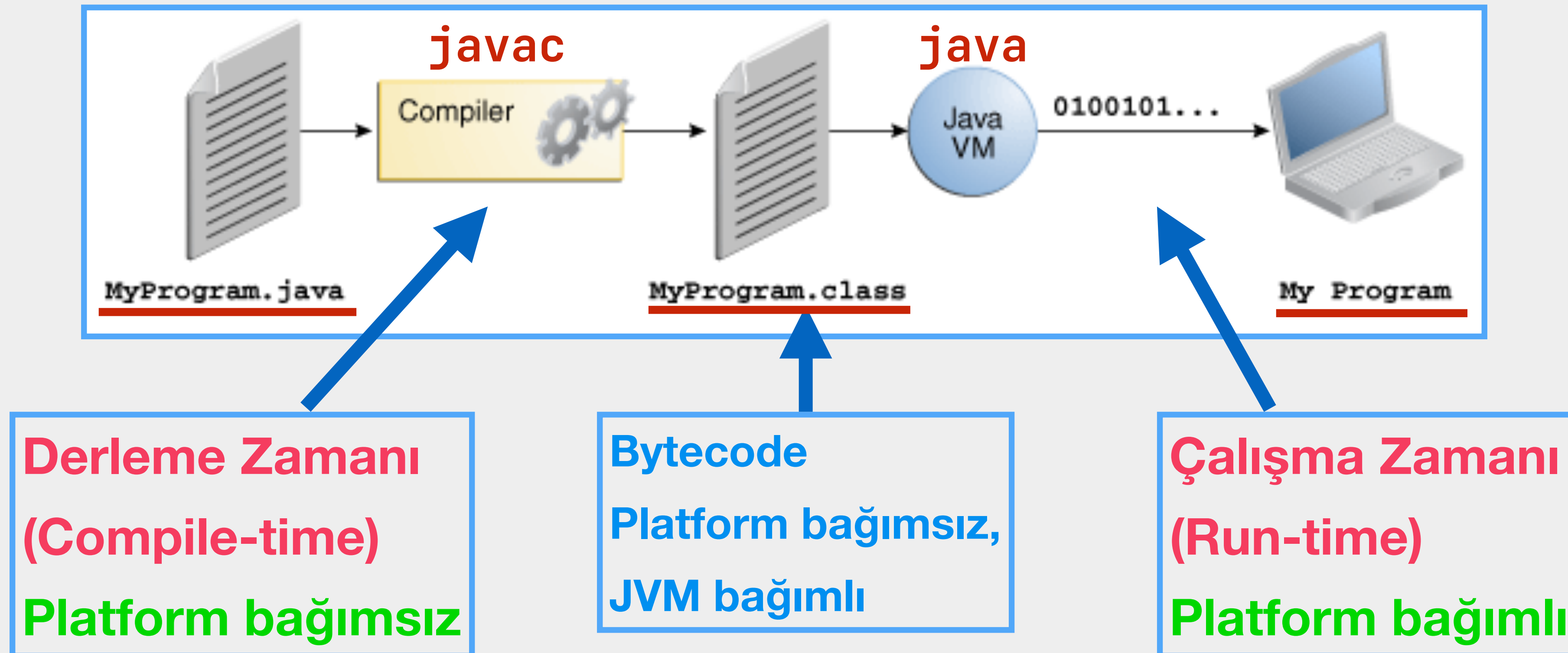
# Java Derlenir mi Yorumlanır mı?

# Java Derlenir ve Yorumlanır



- Java hem derlenir hem yorumlanır (Java is both compiled and interpreted)
- Java kaynak kodları (`java` uzantılı dosyalar) doğrudan makina koduna derlenmez, **bytecode**lardan oluşan `class` uzantılı dosyalara derlenir
- Derleme sonucunda oluşan bytecodelar platformdan bağımsızdır ve çalışma zamanında JVM tarafından yorumlanır
- Bu şekilde Java'nın her iki dünyanın da en iyi taraflarını (best of the both worlds) bir araya getirmesi amaçlanmıştır

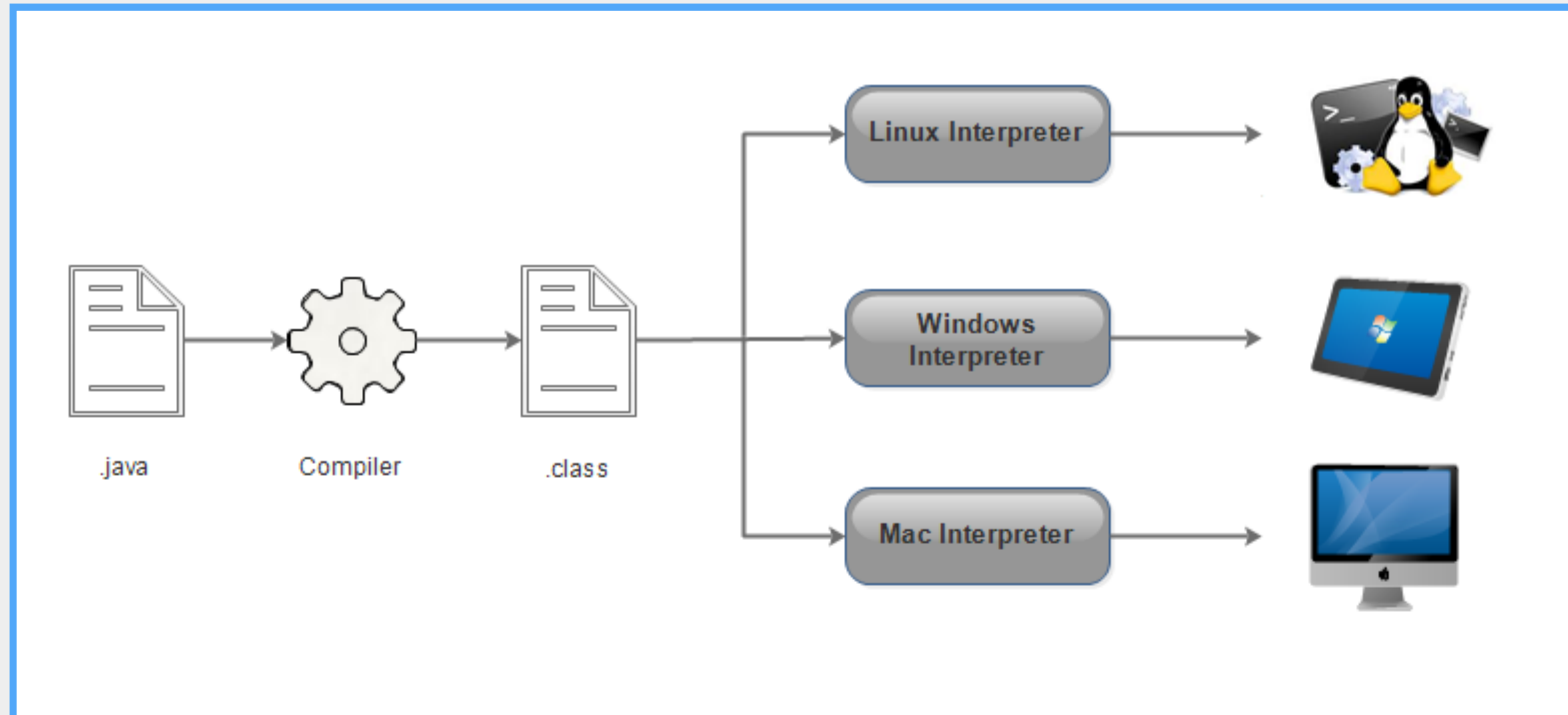
# Derleme ve Yorumlama Süreci



# Bytecodeların Yorumlanması



- Bytecodelardan oluşan `class` dosyası, platformdan bağımsızdır ve JVM'in olduğu herhangi bir platformda JVM tarafından yorumlanır yani çalıştırılır



# Java Kaynak Kodunu Derleme



- Java'da kaynak kodunu (source code) derlemek (compile) için, Java derleyicisi (Java compiler) kullanılır
- Java derleyicisi `javac` komutu ile çalıştırılır
- Java derleyicisi `java` uzantılı kaynak kodundan `class` uzantılı bytecode dosyası üretir
- Eğer aynı isimli `class` dosyası zaten varsa dosya yenilenir

# Selam :)



- Biz de ilk bölümde `Selam :)` yazan bir kaç farklı kodu `java` komutuyla mesela `java Selam.java` ile çalıştırdık ama derlemedik çünkü bir `class` dosyası oluşturmadık!
- Basit denemelerin kolayca yapılabilmesi için, tek dosyadan oluşan Java programlarını derlemeden doğrudan `java` komutu ile çalıştırabilme özelliği Java 11 ile geldi, sonra 25 ile daha da gelişti
- Ama şimdi derleme ve çalıştırma döngüsünü aynı kodlarla yapmayı öğreneceğiz

# SimpleSelam.java'yı Derleme



- Şimdi `javac` ile `ch02/selam` dizinindeki `SimpleSelam.java` dosyasındaki kaynak kodu derleyelim: `javac SimpleSelam.java`
- Bu komutlar kaynak dosyalarının olduğu dizinde çalıştırılmalıdır ve derleme hatası alınmazsa aynı dizinde `SimpleSelam.class` dosyası oluşacaktır

```
public class SimpleSelam{  
    public static void main(String[] args) {  
        System.out.println("Selam :)");  
    }  
}
```

# Bytecode - I



- `class` dosyası bytecodelardan oluşur
- **Bytecodelar, JVM'in komutlarıdır**
  - Bir byte uzunluğunda oldukları için böyle adlandırılmıştır
- Bytecode, herhangi bir platforma özel değildir ya da başka bir ifadeyle sadece JVM'e özeldir

# Bytecode - II



- Bytecode'lar, JVM tarafından çalışma zamanında platforma özel makina dili komutları çağrılarak ya da makina diline çevirilerek çalıştırılır
- Bu anlamda Java için platform, JVM'in kendisidir
- Bu şekilde Java'nın platformdan bağımsız ve taşınabilir olması sağlanır

# SimpleSelam.class'ı Çalıştırma - I



- Java'da ancak içinde `main` metot olan sınıflar çalıştırılabilir (*executable*, *runnable*) ya da daha doğru ifadeyle, `main` metotlu sınıfların `class` dosyaları çalıştırılabilir
- `SimpleSelam.java`'da yani `SimpleSelam.class`'da `main` metot vardır
- Dolayısıyla `SimpleSelam` sınıfını `java` komutuna geçerek JVM'i başlatmalıyız
- İçinde Java yorumlayıcısını barındıran JVM, `SimpleSelam` sınıfını yükler, çalıştırır ve çıktısını komut satırına basar

# SimpleSelam.java'ı Derleme ve Çalıştırma



```
akin@M3-Max:~/Desktop/Code/ch02/selam
/Users/akin/Desktop/Code/ch02/selam >ls -l
total 24
-rwxrwxrwx@ 1 akin  staff  265 22 Şub  2025 Selam.java
-rwxrwxrwx@ 1 akin  staff  396 13 Haz  11:51 SelamTest.java
-rw-r--r--@ 1 akin  staff  162 31 Ağu  00:44 SimpleSelam.java
/Users/akin/Desktop/Code/ch02/selam >javac SimpleSelam.java
/Users/akin/Desktop/Code/ch02/selam >ls -l
total 32
-rwxrwxrwx@ 1 akin  staff  265 22 Şub  2025 Selam.java
-rwxrwxrwx@ 1 akin  staff  396 13 Haz  11:51 SelamTest.java
-rw-r--r--  1 akin  staff  425 31 Ağu  00:44 SimpleSelam.class
-rw-r--r--@ 1 akin  staff  162 31 Ağu  00:44 SimpleSelam.java
/Users/akin/Desktop/Code/ch02/selam >java SimpleSelam
Selam :)
/Users/akin/Desktop/Code/ch02/selam >
```

Kaynak kod dosyası

Derleme

Üretilen class dosyası

Çalıştırma/Yorumlama

# JVM, Java Virtual Machine - I

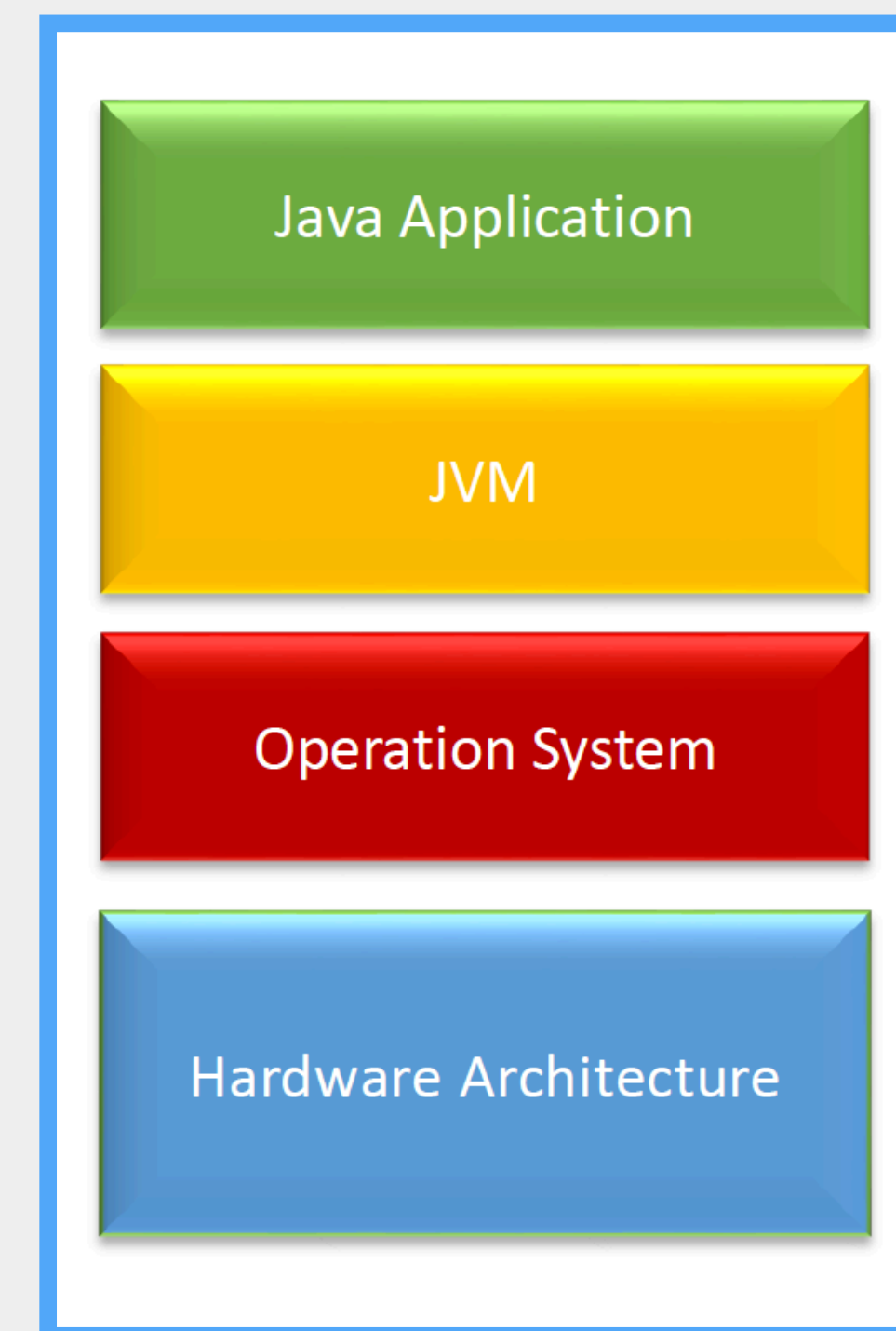
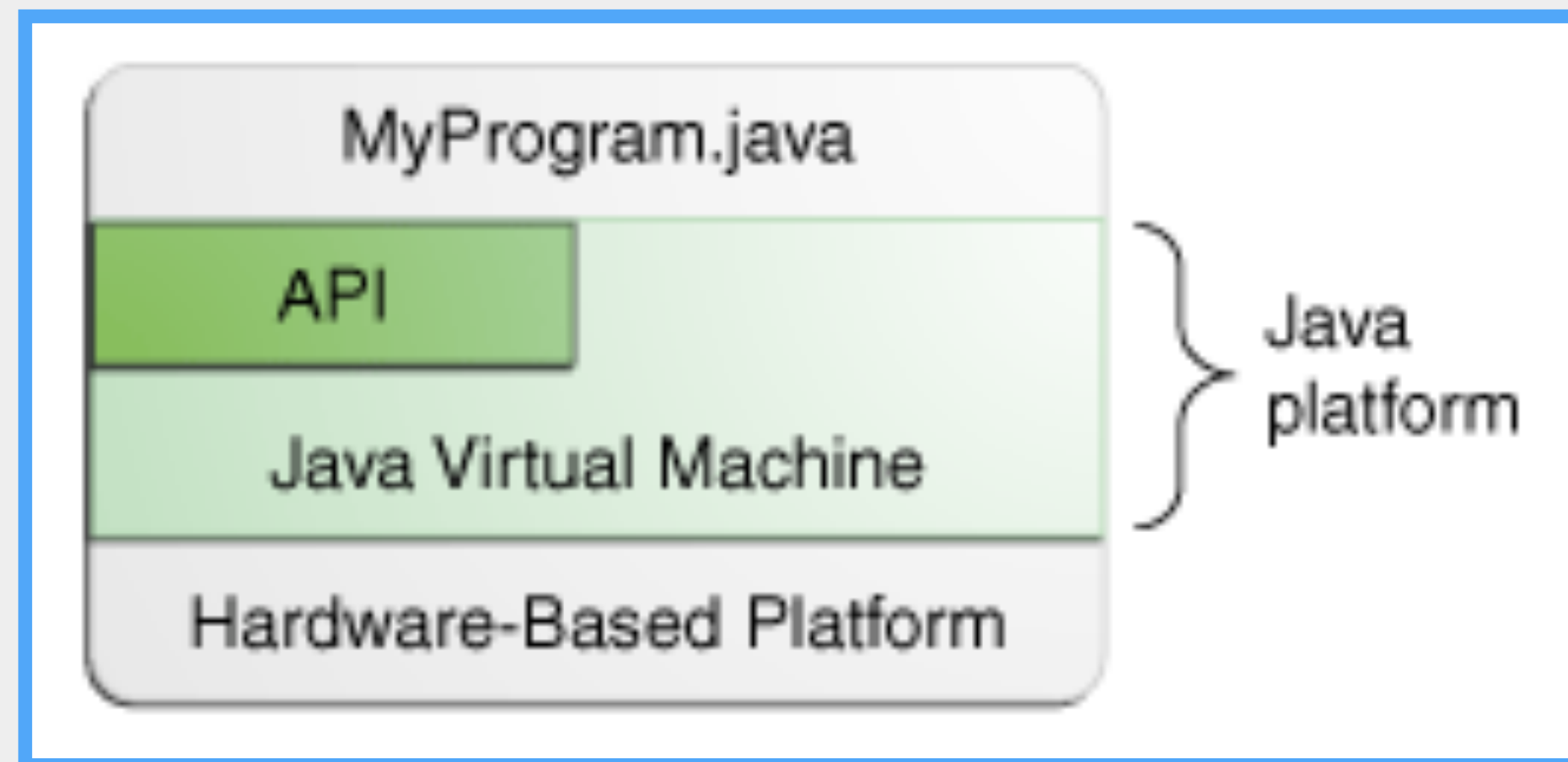


- JVM (Java Virtual Machine ya da Java Sanal Makinası), sanal bir makinadır ya da platformdur
- Derlenmiş Java kodlarını yani bytecodeları çalıştıran sistemdir
- Genelde içinde *Just-in-Time Compiler* barındırır
- JRE'nin (Java Runtime Environment) ana parçasıdır
- Genel olarak C/C++ ve Java ile yazılır
- Java'nın çalışması beklenen her platformda kurulu olmalıdır

# JVM, Java Virtual Machine - II



- JVM'i, geliştiriciye bakan yüzü standart olan ama platforma bakan yüzü, onunla konuşabilecek şekilde, geliştiriciyi platformdan soyutlayan bir ara katman, ara platform olarak düşünebilirsiniz



# SimpleSelam.java'ı Derlemeden Çalıştırma



- İlk bölümde Java kaynak kodlarımızı `java` komutuyla derlemeden çalıştırmıştık, o zaman henüz derleme ve yorumlama farkını bilmiyorduk, artık biliyoruz ve az önce kodumuzu derleyip çalıştırdık
- İlk bölümdeki gibi yaparak `SimpleSelam.java` kaynak kodunu derlemeden çalıştırabiliriz:

```
java SimpleSelam.java
```

- Bu durumda derleme işi bellekte yapılır ve `class` dosyası oluşturulmaz

# Hata Durumunda - I



- Java kaynak kodu `javac` ile derlerken alınan hataya **derleme hatası** (**compilation error**) denir
- Hata olduğunda `class` dosyası oluşmaz, varsa yenilenmez, kaynak kodundaki hatanın giderilip derlemenin tekrarlanması gerekir
- Bu eğitimde “olamaz”, “hatadır” vb. şeklindeki ifadeler aksi söylenmedikçe, belirtilen kurala uyulmadığında derleme hatası alınacağı anlamına gelmektedir

# Hata Durumunda - II



- Hatayı bulmak için noktalama işaretlerine, örneğin `String` sınıfının isminin büyük harfle başlaması gibi büyük-küçük harf ayırımlarına ya da `String` nesnelerinin iki tane çift tırnak arasında olması gibi kurallara dikkat edin ve kodunuzun buradakiyle aynı olmasını sağlayın
- İleride göreceğiz, her dilde olduğu gibi Java'da da uyulması gereken söz dizimi (syntax) kuralları vardır
- Ve Java'da küçük-büyük harf ayırımı vardır

# SimpleSelam.java'ı Derlemede Hata



```
akin@M3-Max:~/Desktop/Code/ch02/selam
/Users/akin/Desktop/Code/ch02/selam >ls -l
total 24
-rwxrwxrwx@ 1 akin  staff  265 22 Şub  2025 Selam.java
-rwxrwxrwx@ 1 akin  staff  396 13 Haz  11:51 SelamTest.java
-rw-r--r--@ 1 akin  staff  161 31 Ağu  00:51 SimpleSelam.java
/Users/akin/Desktop/Code/ch02/selam >cat SimpleSelam.java
// Exactly the same as Selam.java in ch01

public class SimpleSelam{
    public static void main(String[] args) {
        System.out.println("Selam :) );
    }
}%

/Users/akin/Desktop/Code/ch02/selam >javac SimpleSelam.java
SimpleSelam.java:5: error: unclosed string literal
        System.out.println("Selam :) );
                           ^
1 error
/Users/akin/Desktop/Code/ch02/selam >
```

**Hatalı String**

**SimpleSelam.java**

**Derleyici/derleme hatası  
(Compiler/compilation error)**

# Selam.class'ı Çalıştırmada Hata



- Eğer `main` metoda sahip olmayan bir sınıfı çalıştırmak isterseniz yine hata alırsınız:

```
akin@M3-Max:~/Desktop/Code/ch02/selam
~/Users/akin/Desktop/Code/ch02/selam > javac SelamTest.java
~/Users/akin/Desktop/Code/ch02/selam > java SelamTest
Selam Ayşe :)
Selam Sinan :)
~/Users/akin/Desktop/Code/ch02/selam > java Selam
Error: Main method not found in class Selam, please define the main method as:
    public static void main(String[] args)
or a JavaFX application class must extend javafx.application.Application } Hata!
~/Users/akin/Desktop/Code/ch02/selam > java SelamTest.java
Selam Ayşe :)
Selam Sinan :)
~/Users/akin/Desktop/Code/ch02/selam > java Selam.java
error: can't find main(String[]) method in class: Selam } Hata!
~/Users/akin/Desktop/Code/ch02/selam >
```

- Bu, ister derlenmiş `class` dosyası olsun ister doğrudan Java kaynak kodu çalıştırılsın her zaman hatadır

# Bytecode - III



- `javap`, JDK ile birlikte gelen ve `class` dosyasının içeriğini yani bytecodelerini gösteren bir araçtır
- Aşağıdaki komutla `SelamTest.class` dosyasının bytecodelerini görebilirsiniz

```
javap -c SelamTest.class
```

Compiled from "SelamTest.java"

```
public class SelamTest {
```

```
    public SelamTest();
```

```
        Code:
```

```
            0: aload_0
```

```
            1: invokespecial #1
```

```
                // Method java/lang/Object."<init>": ()V
```

```
            4: return
```

```
    public static void main(java.lang.String[]);
```

```
        Code:
```

```
            0: new           #7
```

```
                // class Selam
```

```
            3: dup
```

```
            4: invokespecial #9
```

```
                // Method Selam."<init>": ()V
```

```
            7: astore_1
```

```
            8: aload_1
```

```
            9: ldc           #10
```

```
                // String Ayşe
```

```
           11: invokevirtual #12
```

```
                // Method Selam.selamSöyle: (Ljava/lang/String;)Ljava/lang/String;
```

```
           14: astore_2
```

```
           15: getstatic     #16
```

```
                // Field java/lang/System.out:Ljava/io/PrintStream;
```

```
           18: aload_2
```

```
           19: invokevirtual #22
```

```
                // Method java/io/PrintStream.println: (Ljava/lang/String;)V
```

```
           22: new           #7
```

```
                // class Selam
```

```
           25: dup
```

```
           26: invokespecial #9
```

```
                // Method Selam."<init>": ()V
```

```
           29: astore_1
```

```
           30: aload_1
```

```
           31: ldc           #28
```

```
                // String Sinan
```

```
           33: invokevirtual #12
```

```
                // Method Selam.selamSöyle: (Ljava/lang/String;)Ljava/lang/String;
```

```
           36: astore_2
```

```
           37: getstatic     #16
```

```
                // Field java/lang/System.out:Ljava/io/PrintStream;
```

```
           40: aload_2
```

```
           41: invokevirtual #22
```

```
                // Method java/io/PrintStream.println: (Ljava/lang/String;)V
```

```
           44: return
```

```
}
```

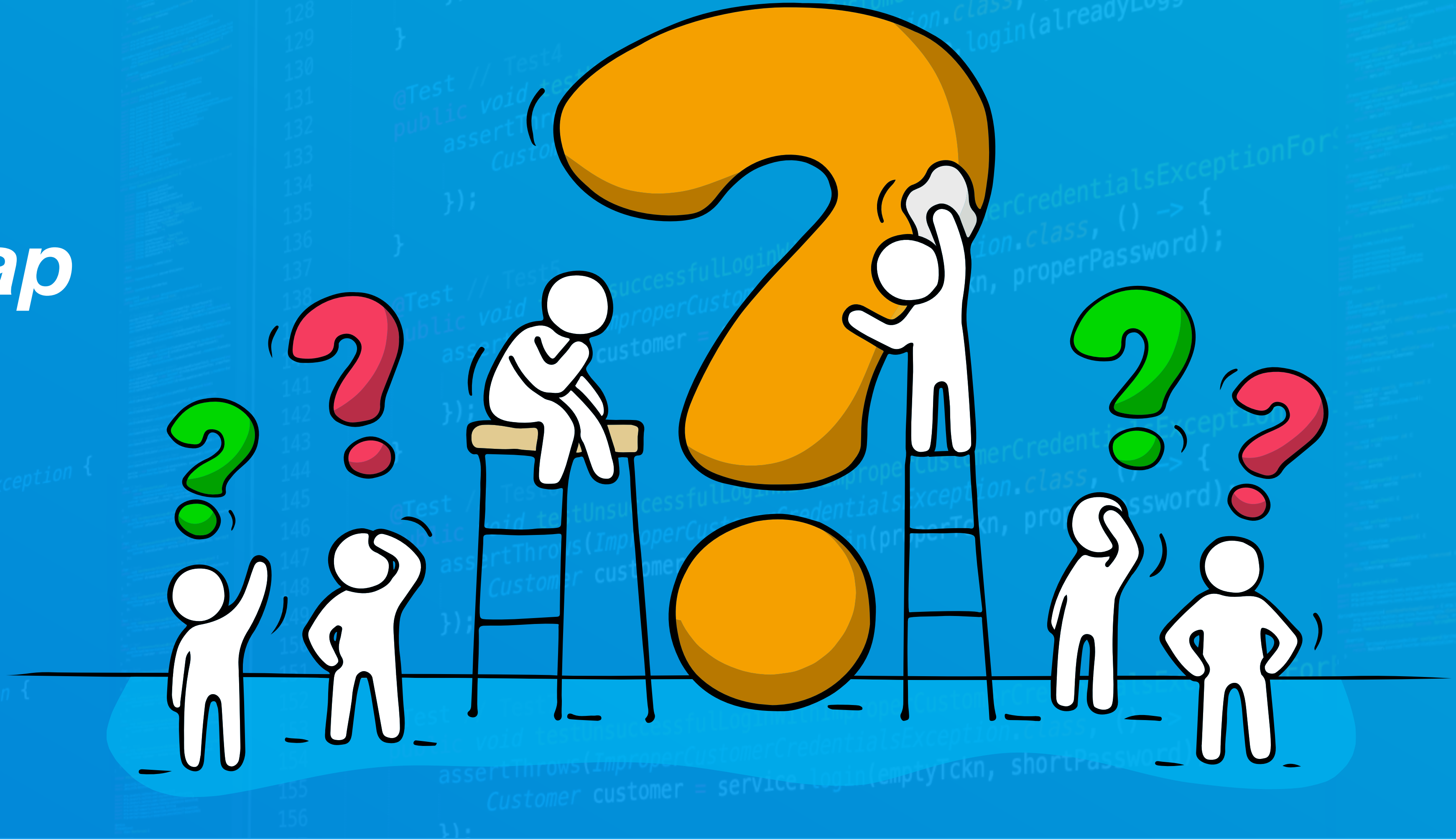


# Ödev



- Verilen `MonteCarloPI.java` dosyasını kullanarak `MonteCarloPI` sınıfını önce derleyin sonra da çalıştırın
- Aynı sayıda nokta kullanarak, C++, Python ve Java arasında performans kıyaslaması yapın

# Soru ve Cevap Zamanı!





# Java Nesne Merkezlidir

# Java Nesne Merkezlidir - I



- **Java bir nesne merkezli programlama dilidir (object-oriented programming language)**
- Nesne merkezli (nesne yönelimli) diller, prosedürel/yordamsal (procedural/imperative) ya da fonksiyonel (functional) dillerden farklıdır
- Prosedürel dillerde en önemli kavram/soyutlama, yordam/prosedür/fonksiyondur ve bu yaklaşım, bir problemi alt problemlere bölüp, her birini adım adım (step-wise decomposition) tanımlayarak çözer
- Fakat nesne merkezli diller, nesne kavramı üzerine otururlar

# Java Nesne Merkezlidir - II



- Nesne, insan zihninin kendisine yöneldiği, özellik ve davranışlara sahip, fiziksel ya da kavramsal herhangi bir şeydir
- Yazılımda her tip nesne için, verisi ile davranışını bir araya getirip sarmalayan ya da kapsülleyen (encapsulation) ve adına genelde **sınıf** (**class**) denen kalıplar (şablonlar) oluşturulur

**Sınıf = Veri + Davranış**

- Nesneler, sınıflardan üretilmiş ve onların birer örneği (instance) olan çalışma zamanı yapılarıdır

# Nesne Merkezli Programlama



- Dolayısıyla nesne merkezli programlamada (NMP) (object-oriented programming, OOP), tüm problemler, problemdeki nesneler ve aralarındaki ilişkiler üzerinden modellenir
- Java'da nasıl NMP yapacağımızı birazdan bir örnekle göreceğiz ama önce ufak bir hatırlatma yapalım

# Açıklama - I



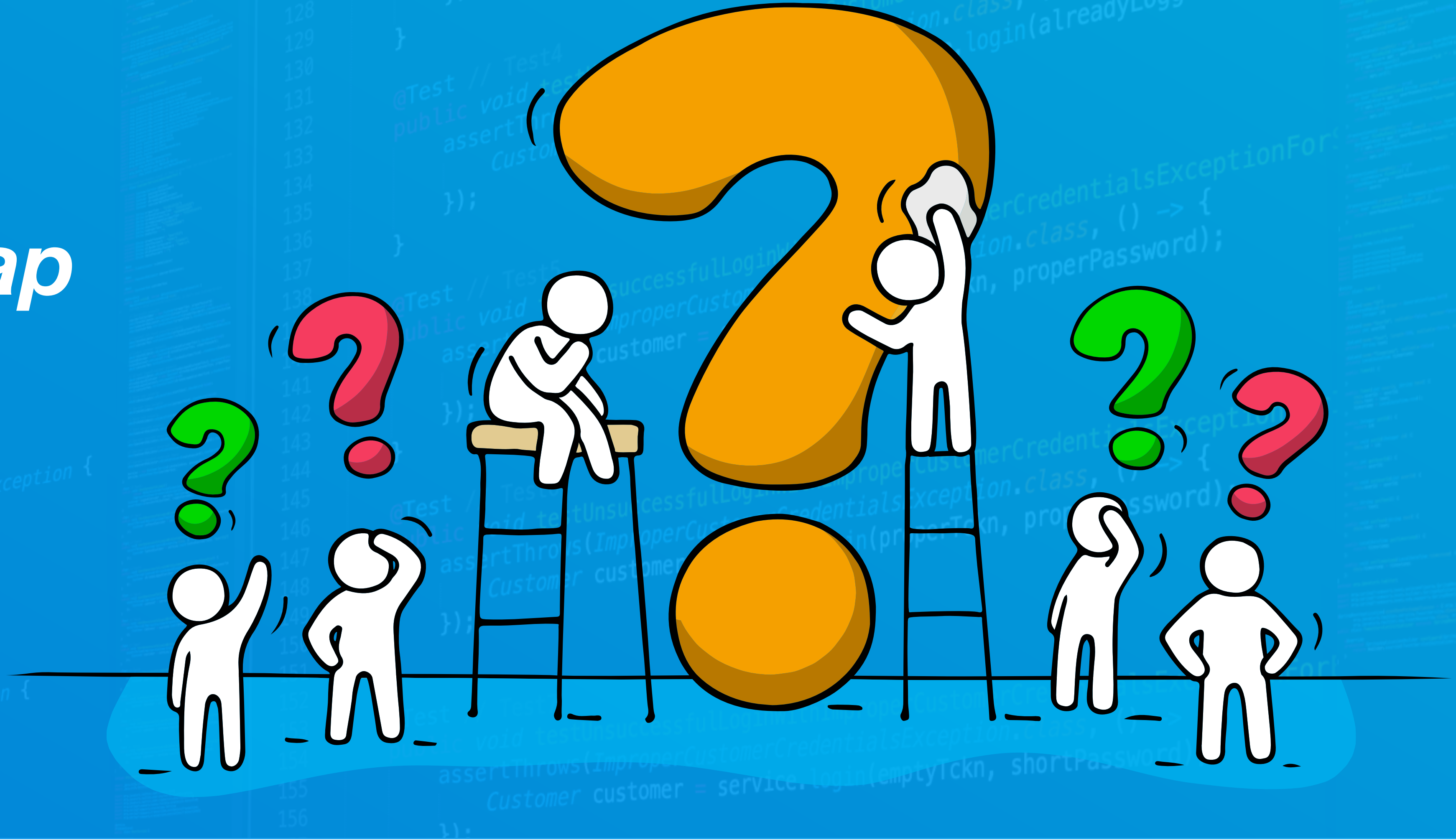
- **Object-oriented** terimi dilimize yaygın olarak **Nesne Yönelimli** olarak çevrilmektedir
- Buna bir itirazım olmamakla birlikte ben **Nesne Merkezli** demeyi tercih ediyorum
- Nesne Merkezli teriminin, nesneyi merkeze alan yaklaşımın ruhunu daha iyi aktardığını düşünüyorum
- Nesne Yönelimli ya da Nesne Merkezli diyebilirsiniz

# Açıklama - II



- Fakat **Object-Oriented** terimi zaman zaman dilimize *nesne tabanlı* olarak da çevriliyor
- Nesne tabanlı, *object-based*'in karşılığıdır ve daha ilkel bir nesne soyutlamasına sahip dillere verilen genel bir addır
- Dillerin tarihinde nesne tabanlı diller olmuştur
- Dolayısıyla C++, Java, C#, Go, nesne-tabanlı değil, nesne merkezli ya da nesne yönelimlidir

# Soru ve Cevap Zamanı!





# İlk Nesne Merkezli Kod

# İlk Java Kodumuz



- Daha önce `Selam :)` örneklerini yazıp önce `java` ile çalıştırdık sonra `javac` ile derleyip `java` ile çalıştırdık

```
public class SimpleSelam{  
    public static void main(String[] args) {  
        System.out.println("Selam :)");  
    }  
}
```

- Ama hiç nesne oluşturmadık dolayısıyla NMP yapmadık
- Şimdi `Selam :)` örneğini bu sefer nesne kullanarak tekrar yapalım
- Ama siz detaylara değil NMP'ya odaklanın, detaylar zamanla öğrenilecektir

# Selam - I



- Bu yeni kod örneğinde, `Selam` sınıfı ve bu sınıfta da bir nesne metodu (object method), `selamSöyle()` bulunmaktadır
- `selamSöyle()` metodu `String` tipinde bir değer almakta ve buna `kime` değişkeniyle ulaşmaktadır
- `selamSöyle()` metodu `kime` değişkenini kullanarak yeni bir `String` değer, selamlama cümlesi, üretmekte ve bu değeri geriye, çağıran yere döndürmektedir

```
public class Selam{  
    public String selamSöyle(String kime){  
        return "Selam " + kime + " :)";  
    }  
}
```

# Selam - II



- Örnekteki ikinci sınıf `SelamTest`'tir ve içindeki `main` metotta önce `Selam` sınıfının bir nesnesi oluşturulmakta
- Sonra da `String` tipinde bir değer geçilerek bu nesne üzerindeki `selamSöyle()` metodu çağrılmakta ve dönen cevap yazdırılmaktadır
- İçinde `main` metot olan `SelamTest`'i IDE'de çalıştırabilirsiniz

```
public class Selam{  
    public String selamSöyle(String kime){  
        return "Selam " + kime + " :)";  
    }  
}
```

```
public class SelamTest{  
    public static void main(String[] args){  
        Selam nesne = new Selam();  
        String cevap = nesne.selamSöyle("Ali");  
        System.out.println(cevap);  
    }  
}
```

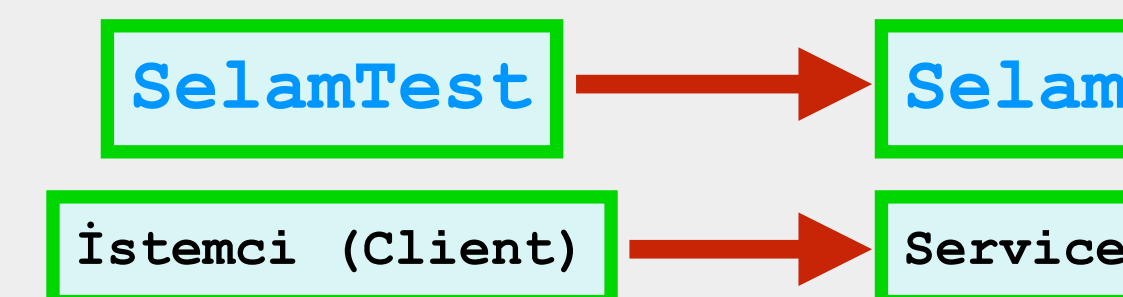
# Selam - III



- `Selam` sınıfını, `selamSöyle()` metoduyla hizmet veren bir servis (service) olarak düşünebilirsiniz
- Aslında hizmet veren `Selam` sınıfının nesneleridir, kendisi değil! Dolayısıyla `selamSöyle()` de bir nesne metodudur
- `SelamTest` sınıfını da `main` metodunda `Selam` sınıfının nesnesinden, onun `selamSöyle()` metodunu çağırarak hizmet alan istemci (client) olarak düşünebilirsiniz

```
public class Selam{  
    public String selamSöyle(String kime){  
        return "Selam " + kime + " :)";  
    }  
}
```

```
public class SelamTest{  
    public static void main(String[] args){  
        Selam nesne = new Selam();  
        String cevap = nesne.selamSöyle("Ali");  
        System.out.println(cevap);  
    }  
}
```



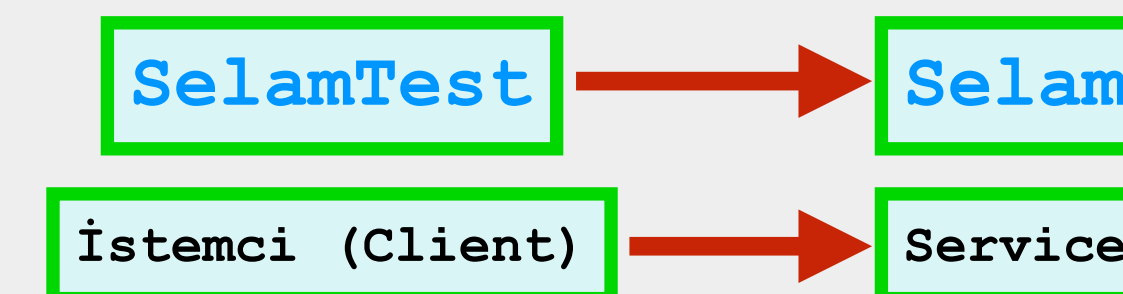
# Selam - IV



- Çünkü, `Selam` sınıfının `selamSöyle()` metodu, kendisine geçilen ismi kullanarak bir selamlama cümlesi üretmekte ve bu cümleyi `SelamTest` sınıfının `main` metoduna geri döndürmektedir
- Üretilen selamlama cümlesinin bir metin olduğuna ve bu sebeple tipinin `String` olduğuna dikkat edin

```
public class Selam{  
    public String selamSöyle(String kime){  
        return "Selam " + kime + " :)";  
    }  
}
```

```
public class SelamTest{  
    public static void main(String[] args){  
        Selam nesne = new Selam();  
        String cevap = nesne.selamSöyle("Ali");  
        System.out.println(cevap);  
    }  
}
```



# Selam - V



- Bu örnekleri IDE'de çalıştırdık
- Daha önce yaptığımız gibi kaynak kod dosyalarını masa üstünde `Code\ch02\selam` dizini altına kopyalayıp komut satırından da çalıştırabiliriz
- Bunun için tek dikkat etmemiz gereken şey, kaynak kodların başındaki **package** anahtar kelimesiyle başlayan satırı silmektir
- **package** yapısını görünceye kadar komut satırında çalıştıracağımız kodlarımız **package** cümlesine sahip olmayacak

# Selam - VI



- Aralarındaki ilişkiden dolayı sırayla, önce `Selam` sonra da `SelamTest` sınıfını derleyeceğiz:
- `javac Selam.java` ve `javac SelamTest.java`
- Bu komutlar kaynak dosyalarının olduğu dizininde çalıştırılmalıdır ve sonucunda aynı dizinde sırayla `Selam.class` ve `SelamTest.class` dosyaları oluşacaktır
- Aslında `javac SelamTest.java` ile hem `SelamTest.java` hem de aralarındaki bağımlılıktan dolayı `Selam.java` derlenir

# Selam - VII



- Sonrasında `main` metoda sahip olan `SelamTest` sınıfını çalıştırabiliriz:

```
java SelamTest
```

- Unutmayın, `SelamTest`, `SelamTest.java` kaynak kodundaki sınıfın ismidir, `SelamTest.class` dosyasının ismi değildir!
- Bunu daha sonra detaylıca göreceğiz

# Selam - VIII



- Ayrıca Java 23 ile gelen özellekle, birden fazla kaynak dosyası olsa da derleme yapmadan doğrudan `java` komutu ile `main` metotlu bir sınıfı çalıştırabilirsiniz: `java SelamTest.java`

# X ve XTest Sınıfları



- Sıklıkla örnek kodlarımızda bu şekilde **X** ve **XTest** sınıfları göreceksiniz
- Anlamak istediğimiz kavram ya da mekanizma **X** sınıfında, bu kavram ya da mekanizmayı test eden kodlar da **XTest** sınıfında olacaktır
- **X** sınıfını hizmet sağlayıcı service, **XTest** sınıfını da ondan hizmet alan istemci (client) olarak düşünebilirsiniz

```
public class XTest{  
    public static void main(String[] args){...  
    }  
}
```

```
public class X{  
    ...  
}
```

XTest

X

İstemci (Client)

Service

# Hello



- Aynı örneği `Hello` sınıfıyla da yazabiliriz ve bu sefer üzerinde `sayHello()` metodu olur ve `HelloTest` sınıfı da `main` metodunda nesnesini oluşturup nesne üzerinde `sayHello()` metodunu `String` bir argümanla çağırır
- Bunu da hem IDE'de hem de komut satırında derleyip çalıştırabilirsiniz

```
public class Hello{  
    public String sayHello(String whom){  
        return "Hellooww " + whom + " :)" ;  
    }  
}
```

```
public class HelloTest{  
    public static void main(String[] args){  
        Hello object = new Hello();  
        String answer = object.sayHello("Maria");  
        System.out.println(answer);  
    }  
}
```



- Bu iki örnekte de önce, kendisinden nesne yaratacak bir şablon olarak birer sınıf (`class`) oluşturduk, `Selam` ve `Hello`, sonra bu sınıflara birer metot koyduk, `selamSöyle()` ve `sayHello()` metotları
- Bu metotlar birer nesne metodudur yani sadece nesneler üzerinde çağrılabilir
- Dolayısıyla `SelamTest` ve `HelloTest` test sınıflarının `main` metotlarında `Selam` ve `Hello` sınıflarının nesnelerini oluşturup üzerlerinde bu metotları çağırdık



- Görüldüğü gibi Nesne Merkezli Programlama'nın (NMP) en temel iki yapısı, sınıf ve nesnedir
- Sınıf bir şablon, nesne ise o şablondan üretilen yapıdır, o sınıfın örneğidir (instance)
- Nesne Merkezli olarak geliştirilen programlar da nesnelerin üzerinde metot çağrısı yaparak ilerler

# Greeting - I



- **Selam** ve **Hello** sınıflarının yaptığı işi aslında **Greeting** isimli tek bir sınıfta yapabiliriz
- Bu şekilde farklı dillerde selam verebiliriz
- Bu amaçla **Greeting** sınıfında bir nesne değişkeni (object variable) tutarız ve her **Greeting** nesnesi için ona farklı bir değer atayabiliriz

```
public class Greeting{
    String greetingWord;

    public String greet(String whom){
        return greetingWord + " " + whom + " :)" ;
    }
}
```

```
public class GreetingTest{
    public static void main(String[] args){
        Greeting selamlama = new Greeting();
        selamlama.greetingWord = "Selam";
        String answer = selamlama.greet("Zeynep");
        System.out.println(answer);

        Greeting greeting = new Greeting();
        greeting.greetingWord = "Hellooww";
        String answer = greeting.greet("Tom");
        System.out.println(answer);
    }
}
```

# Greeting - II



- `Greeting`'deki `greet()` isimli nesne metodu hem `String` tipinde geçilen argümanı hem de nesne değişkeni olan `greetingWord`'u kullanarak bir selamlama cümlesi üretip geri döndürür
- `GreetingTest` de `main` metodunda nesneler oluşturup bu nesnelerin `greetingWord` değişkenlerine farklı dillerde değerler verir ve sonra da argüman geçerek `greet()` metodunu çağırır

```
public class Greeting{
    String greetingWord;

    public String greet(String whom){
        return greetingWord + " " + whom + " :)" ;
    }
}
```

```
public class GreetingTest{
    public static void main(String[] args){
        Greeting selamlama = new Greeting();
        selamlama.greetingWord = "Selam";
        String answer = selamlama.greet("Zeynep");
        System.out.println(answer);

        Greeting greeting = new Greeting();
        greeting.greetingWord = "Hellooww";
        String answer = greeting.greet("Tom");
        System.out.println(answer);
    }
}
```

# Greeting - III



- Bu örnekte, `greetingWord` ve `greet()`'in `Greeting` sınıfında tanımlanan ve `Greeting` sınıfının nesnelere ait özellik (property) ve davranış (behavior) olduğunu anlayabiliriz
- Bundan dolayı `main` metotta farklı nesneler oluşturup onların `greetingWord` nesne değişkenine farklı değerler atadık ve bu şekilde farklı dillerde selam cümleleri ürettik

```
public class Greeting{
    String greetingWord;

    public String greet(String whom){
        return greetingWord + " " + whom + " :)" ;
    }
}
```

```
public class GreetingTest{
    public static void main(String[] args){
        Greeting selamlama = new Greeting();
        selamlama.greetingWord = "Selam";
        String answer = selamlama.greet("Zeynep");
        System.out.println(answer);

        Greeting greeting = new Greeting();
        greeting.greetingWord = "Hellooww";
        String answer = greeting.greet("Tom");
        System.out.println(answer);
    }
}
```



# Ödev

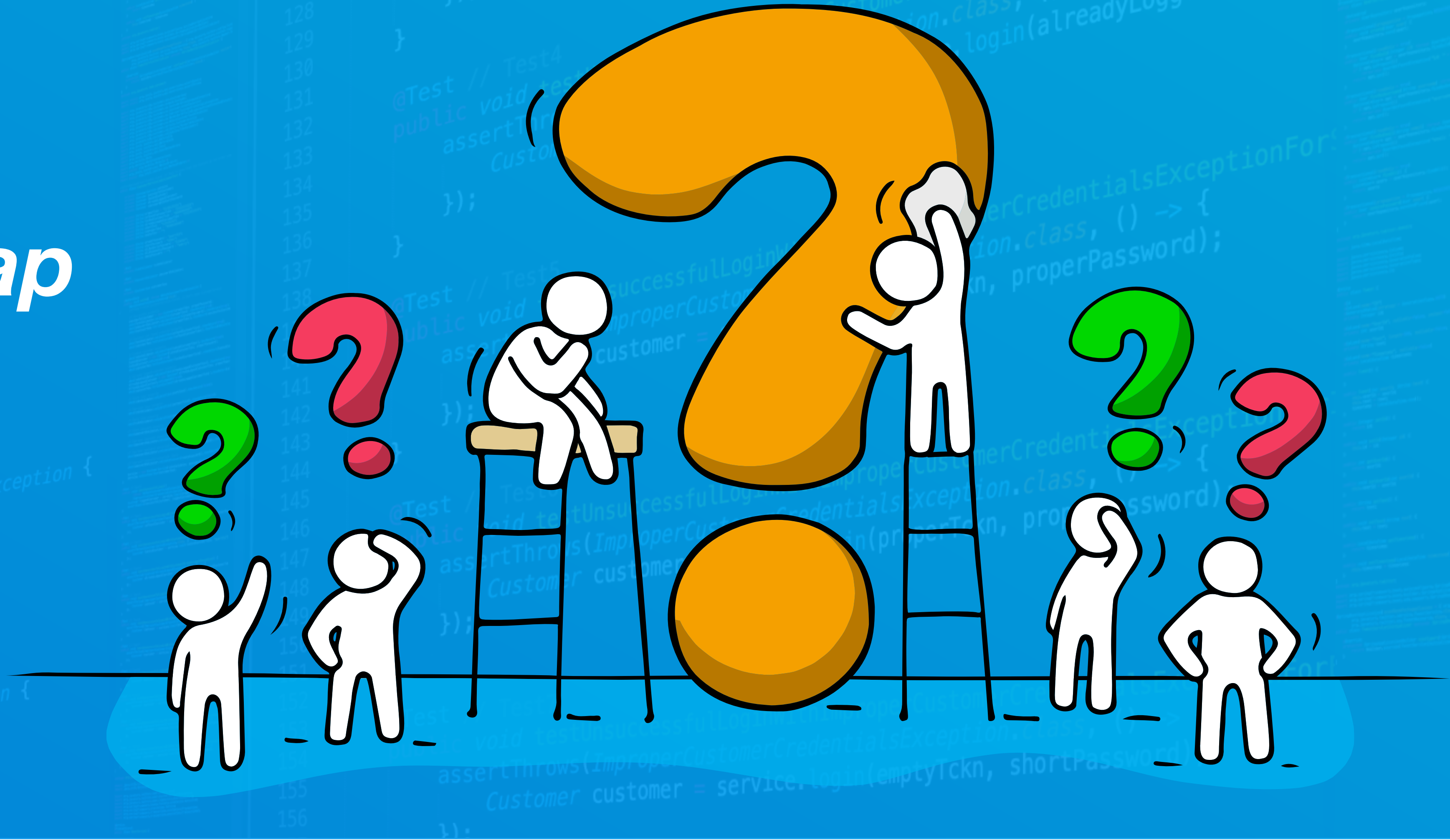


- Siz de kendinizle ilgili isim, soy isim, cinsiyet, yaş vb. bilgileri yazan bir programı şu iki şekilde yazın:
- Doğrudan `main` metotta, `System.out.println()` ya da daha kısa haliyle `IO.println()` kullanarak
- NMP yaklaşımıyla uygun gördüğünüz bir sınıf yaratıp üzerine nesne değişkenleri koyup onları basan bir `printInfo()` metodu yazarak



- Bu durumda bir de test sınıfı yazıp onun **main** metodunda ilk sınıfınızın nesnesini oluşturup üzerindeki değişkenlere değerler atayıp **printInfo()** metodunu çağırmanız gerekecektir
- **printInfo()** metodu, nesnenin değişkenlerine ulaşip değerlerini basacaktır
- Bu örnekte kullanacağınız nesne değişkenlerinin tiplerini **String** yapıp değerlerini çift tırnak içinde, “Fatma” veya “Kadın” şeklinde verebilirsiniz

# Soru ve Cevap Zamanı!





# Bazı Noktalar

# İki Önemli Soru



- Önceki örneklerle ilgili şu iki soru aklımıza takılabilir:
- Neden iki tane sınıf, `Selam` ve `SelamTest`, oluşturduk?
- Neden iki tane kaynak kod dosyası, `Selam.java` ve `SelamTest.java`, oluşturduk?

```
public class Selam{  
    public String selamSöyle(String kime){  
        return "Selam " + kime + " :)";  
    }  
}
```

```
public class SelamTest{  
    public static void main(String[] args){  
        Selam nesne = new Selam();  
        String cevap = nesne.selamSoyle("Ali");  
        System.out.println(cevap);  
    }  
}
```

# Neden İki Sınıf?



- İlk sorunun yani *Neden iki tane sınıf, **Selam** ve **SelamTest**, oluşturduk?* sorusunun cevabı şudur:
- Çünkü verilen bir isme selam üreten nesne ile ondan bu işi yapmasını isteyen nesne yani servis sağlayan ve bu servisi tüketen olmak üzere iki ayrı nesne vardır; bu iki ayrı işi tek nesnede birleştirmek uygun değildir
  - Her ne kadar bu örnekte **SelamTest**'in nesnesi oluşturulmasa da
- Bu cevap soyuttur ve nesne anlayışımız güçlendikçe daha anlaşılır olacaktır

# Neden İki Kaynak Kod Dosyası?



- İkinci sorunun yani *Neden iki tane kaynak kod dosyası, `Selam.java` ve `SelamTest.java`, oluşturduk?* sorusunun cevabı da, birden fazla geliştiricinin bulunduğu ortamda kaynak kod dosyalarının yönetimini daha kolay yapabilmek ile ilgilidir
- Yani dosyaların oluşturulması, değiştirilmesi, bir yerde (repository) saklanması vb. işleri rahat yapabilmek için büyük boyutta az sayıda dosya yerine küçük boyutta çok sayıda dosya kullanmak kolaylık sağlar
- Bu cevap önceki cevaba göre daha anlaşılırdır

# İki Dosya Yerine Tek Dosya - I



- Mantıksal ayırımdan dolayı `Selam` ve `SelamTest` isimli iki sınıf yerine tek bir sınıf oluşturmak çok anlamlı olmasa da bu iki sınıfı tek bir kaynak kod dosyasına koyabiliriz
- Dolayısıyla birden fazla geliştiricinin olmadığını ya da mesela hızlıca tek başımıza örnek bir kod yazdığımızı varsayabiliriz
- Bu durumda `Selam` sınıfı `SelamTest.java` dosyasına konabileceği gibi, iki sınıf da `SelamApp.java` isimli bir kaynak kod dosyası içinde olabilir

# İki Dosya Yerine Tek Dosya - II



- Bu birleştirmede tek dikkat edilmesi gereken nokta, `public` olan sınıfın kendi ismine sahip bir kaynak kod dosyasında bulunmasının zorunlu oluşudur
- Yani `Selam` sınıfı da `SelamTest.java` dosyasına konuyorsa sadece `SelamTest` sınıfı `public` olabilir, `Selam` sınıfı `public` olamaz

```
//SelamTest.java
public class SelamTest{

    public static void main(String[] args){
        Selam nesne = new Selam();
        String cevap = nesne.selamSöyle("Ali");
        System.out.println(cevap);
    }
}

class Selam{
    public String selamSöyle(String kime){
        return "Selam " + kime + " :)";
    }
}
```

# İki Dosya Yerine Tek Dosya - III



- Bu durumda `javac SelamTest.java` komutuyla dosya derlenir ve iki tane `class` dosyası oluşturulur
- `Selam.class` ve `SelamTest.class`
- `java SelamTest` komutu ile `main` metoda sahip olan `SelamTest` sınıfı çalıştırılır
- Ya da derlemeden `java SelamTest.java` ile doğrudan çalıştırılır

```
//SelamTest.java
public class SelamTest{

    public static void main(String[] args){
        Selam nesne = new Selam();
        String cevap = nesne.selamSöyle("Ali");
        System.out.println(cevap);
    }
}

class Selam{
    public String selamSöyle(String kime){
        return "Selam " + kime + " :)";
    }
}
```

# İki Dosya Yerine Tek Dosya - IV



- İki sınıf da `SelamApp.java` gibi, sınıfların isimlerinden farklı bir isme sahip bir kaynak kod dosyasına konursa, bu durumda sınıfların hiçbirisi `public` olamaz
- `javac SelamApp.java` komutu ile dosya derlenir ve `SelamTest` ve `Selam` class dosyaları oluşturulur
- `java SelamTest` komutuyla `main` metoda sahip olan `SelamTest` sınıfı çalıştırılır

```
//SelamApp.java
class SelamTest{

    public static void main(String[] args){
        Selam nesne = new Selam();
        String cevap = nesne.selamSöyle("Ali");
        System.out.println(cevap);
    }
}

class Selam{
    public String selamSöyle(String kime){
        return "Selam " + kime + " :)";
    }
}
```

# İki Dosya Yerine Tek Dosya - V



- Ya da `java SelamApp.java` komutuyla, derlemeden doğrudan `SelamTest` sınıfının `main` metodu çalıştırılır

```
//SelamApp.java
class SelamTest{

    public static void main(String[] args){
        Selam nesne = new Selam();
        String cevap = nesne.selamSöyle("Ali");
        System.out.println(cevap);
    }
}

class Selam{
    public String selamSöyle(String kime){
        return "Selam " + kime + " :)";
    }
}
```

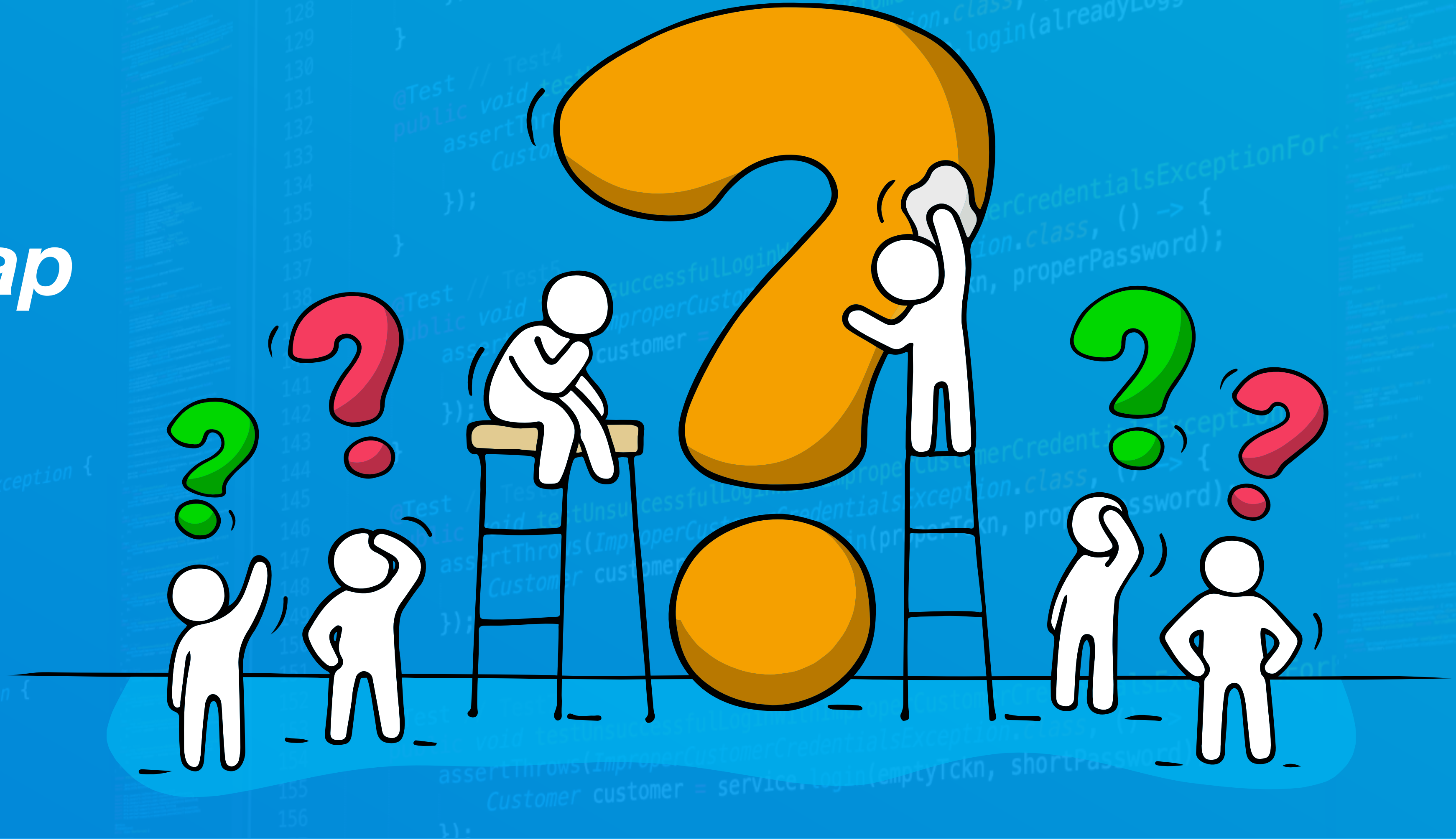


# Ödev



- Bir önceki kendinizle ilgili bilgileri basan programın ikinci halindeki iki sınıfı tek bir dosyaya koyarak derleyip çalıştırın

# Soru ve Cevap Zamanı!



# Java'yı Nasıl Öğrenirim?



# Java'yı Öğrenmek - I



- Java öğrenmeye çekirdek dil yani Java SE ile başlanmalıdır:
  - Java SE bile çok geniştir ve temel söz dizimi (syntax), yapıları (structures) ve kuralları (grammar) ile başlanmalıdır
  - Devamında nesne merkezli programlamanın sınıf yapıları, miras (inheritance), soyut sınıflar (abstract classes), arayüzler (interfaces) ve çok şekillilik (polymorphism) konuları ele alınmalıdır
  - Bu sırada sınıf kütüphanesi, **String** ya da **Date** gibi en yaygın öğelerden başlanarak öğrenilmelidir
  - Sonrasında fonksiyonel programlama (functional programming) öğrenilmelidir

# Java'yı Öğrenmek - II



- Daha sonra dildeki diğer önemli yapılar, sıra dışı durumlar (exceptions), torbalar (collections) ve genel yapılar (generics), giriş-çıkış (input-output, IO), kanallar (threads) ve veri tabanına erişim (JDBC) öğrenilmelidir
- Ayrıca Java API'si bol bol kullanılarak aşinalık elde edinilmelidir
- Java SE'de bu konular dışında, reflection, security, grafik kullanıcı arayüzü (GUI) geliştirme vb. daha pek çok konu vardır ve bunlar da ihtiyaca göre öğrenilebilir
- JDBC ile veri tabanı programlamaya da bu sırada başlanabilir

# Java'yı Öğrenmek - III



- Java SE'de iyi bir noktaya geldikten sonra Java EE'ye geçilebilir
- Muhtemelen Java EE'nin web tarafındaki bileşenleriyle işe başlamak, dolayısıyla JAX-RS ile web API geliştirmeyi öğrenmek, belki ihtiyaca göre JAX-WS ile web servisleri, servlets, JSP ve JSF ile ilerlemek,
- Sonrasında da JPA, transactions, cloud, caching vb. daha ileri konulara geçmek anlamlıdır
- Bunlar öğrenilirken Spring/Spring Boot vb. yaygın kullanılan çerçeveler (frameworks) de ele alınmalıdır



# Java'yı Nereden Öğrenebilirim?

# Java'nın Resmi Sayfası - I



- Java'nın resmi sayfası: <https://www.oracle.com/java/>
- Burada Java ile ilgili temel gelişmelerden haberdar olabilirsiniz
- Ayrıca bu sayfadan, Java ile ilgili tutorial, white paper, makaleler, Java Magazine vb. dokümanlara da ulaşılabilir
- Java'yla ilgili projeleri, yeni Java sürümlerini ve içeriklerini <https://openjdk.org/> sayfasından öğrenebileceğimizi daha önce belirtmiştik

# Java'nın Resmi Sayfası - II



- <https://www.oracle.com/java/technologies/downloads/> adresinde Java'nın, MS Windows, macOS, Linux için dağıtımı bulunduğunu biliyoruz
- Bu sayfanın alt tarafındaki Documentation Download ve Release Information kısmından detaylı Java dokümantasyonuna ulaşabilir ve makinenize indirebilirsiniz
- Buradaki Online Documentation ile o sürümdeki yeniliklere ulaşabilirsiniz

# Java Tutorials - I



- <https://docs.oracle.com/javase/tutorial/> sayfasında **Java Tutorial** vardır
- Bu tutorial JDK 8 için yazılmıştır ve daha sonra gelen yenilikleri içermemektedir
- Ama hala Java'nın temelleri konusunda iyi bir kaynaktır, konuları basit ve açık bir şekilde anlatır
- Örnek kodlarını da indirip çalıştırabilirsiniz
- Bu sayfanın sağındaki linkten Java Tutorial'ı makinalarınıza indirebilirsiniz

# Java Tutorials - II



- Daha yeni kaynaklar için <https://dev.java/> adresine bakabilirsiniz
- Örneğin bu sayfadaki Getting Started with Java (<https://dev.java/learn/getting-started/>) linkiyle JDK kurulumu ve en temel ayarları yanında `Hello, World!` gibi en basit kod örneklerini bulabilirsiniz
- Ya da Java Language Basics (<https://dev.java/learn/language-basics/>) sayfası, akış kontrolüne kadar olan temel konularla ilgili bir tutorialdır

# Java Specifications



- Java dilinin ve JVM'in detaylı tarifi, sahip olması gereken tüm özellikleri specification (spec) denen belgelerde bulunur
- <https://docs.oracle.com/javase/specs/index.html> sayfasında Java 6'dan itibaren tüm spec belgeleri HTML ve PDF olarak vardır
- Spec okumak zordur ama nihai bilgi kaynağı da bu belgelerdir
- Belki Java'da daha fazla bilgi ve tecrübe sahibi oldukça bu specleri takip eder hale gelirsiniz



- Ayrıca Oracle'ın <https://learn.java/> sayfası da özellikle yeni başlayanlar için pek çok değerli içeriğe sahiptir
- Bu sayfadaki Java Playground'u kullanarak Java kodunuzu online yazıp çalıştırabilirsiniz



# Kitaplar



- Aralık 2025 itibariyle tavsiye edeceğim temel yardımcı kitaplar şunlardır:
  - Head-First Java 3rd Ed. (K. Sierra, B. Bates)
  - Java 23 for Absolute Beginners (I. Cosmina)
  - Learning Java 6th Ed. (M. Loy, P. Niemeyer, D. Leuck)
  - Java: A Beginner's Guide 10th Ed (H. Schildt)

# Kitaplardan Faydalanma



- Farkettiğiniz gibi bu kitaplar İngilizce'dir; kitap okuyacak kadar İngilizce'yi bilmelisiniz, bilmiyorsanız Java'yı öğrenirken onu da geliştirmelisiniz
- Bu 4 kitaptan birini, bu eğitimde kendinize ek kaynak olarak belirleyin
  - Bu kitapların PDF vb. formattaki sürümlerine internette ulaşabilirsiniz, inceleyin ve en rahat anladığınız, anlatımı hoşunuza gideni seçin
- Dersi dinleyip anladıktan sonra bu kitaba bakın, konuyu bir de oradan okuyup çalışın; bu sizin daha sağlıklı ilerlemenizi sağlayacaktır

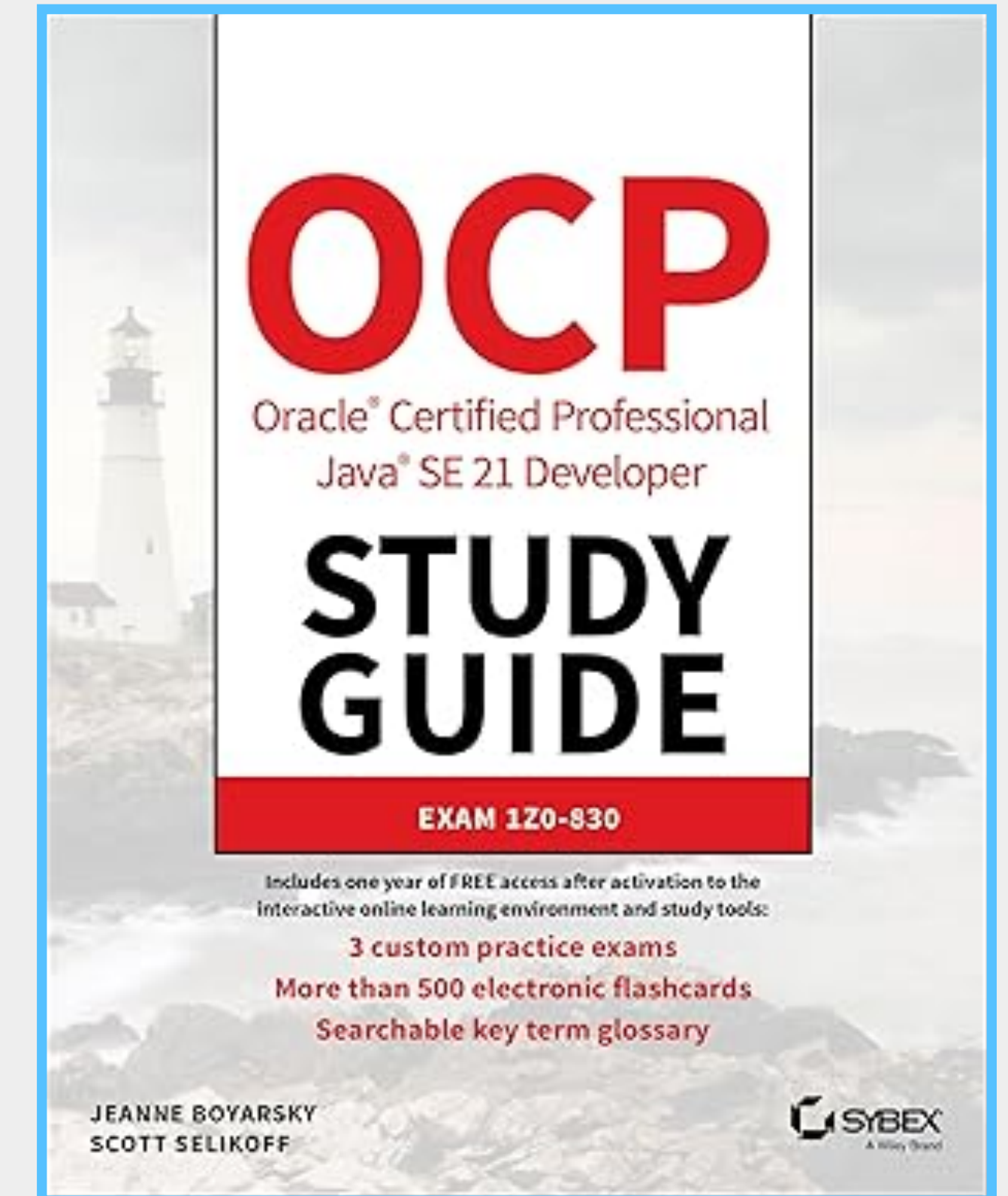


- Ayrıca aşağıdaki iki kitap ta başvuru kaynağı olarak kullanılabilir:
- Core Java 2 cilt 12th Ed. (Cay Hortsman)
- Java The Complete Reference 13th Ed. (Herbert Schildt)
- Bu ansiklopedik kitapların yeni Java sürümleri için güncellenmiş baskıları çıkar
- Dolayısıyla bu kitapları şu an için en baştan öğrenme amacıyla değil de sadece başvuru amacıyla tavsiye edebilirim

# OCA/OCP Sertifikası Hazırlık Kitapları - I



- Aralık 2025 itibariyle henüz Java 25 için OCP sınavı çıkmadı
- Var olan Java 21 OCP sertifikası için en temel kaynak şudur:
- OCP Oracle Certified Professional Java SE 21 Developer Study Guide (J. Boyarsky, S. Selikoff)



# OCA/OCP Sertifikası Hazırlık Kitapları - II



- Java 17 OCP sertifikası için kaynaklar şunlardır:
  - OCP Java SE 17 Developer Study Guide (J. Boyarsky, S. Selikoff)
  - OCP Java SE 17 Practice Tests (S. Selikoff, J. Boyarsky)
- Java 8 OCA ve OCP sertifikası için kaynaklar şunlardır:
  - OCA Java SE 8 Programmer I Study Guide (J. Boyarsky, S. Selikoff)
  - OCA/OCP Java SE 8 Practice Tests (S. Selikoff, J. Boyarsky)

# OCP Sertifikasına Hazırlık - I



- Bir bölümü iyice anlayarak bitirdikten sonra oradaki konularla ilgili OCA/OCP hazırlık soruları çözmeniz Java'da sizi daha da iyi hale getirecektir
- Java 8, 11 ve 17 sertifika sınavına hazırlık kitaplarından da faydalanın, özellikle giriş konuları için OCA hazırlık soruları çok hoşunuza gidecektir
- Artık OCA sınavı ve sertifikası yok, sadece OCP var ama hazırlık için OCA kaynakları da faydalıdır
- OCP sınavını başarıyla geçerseniz, size hem yurt içinde hem de yurt dışında iş bulmanızda yardımcı olacaktır

# OCP Sertifikasına Hazırlık - II



- Bu eğitimde ele alacağımız konular şu anki OCP sertifika sınavının konularını hepsini olmasa da ciddi bir kısmını karşılıyor
- Zaten her yeni OCP sürümünde içerik az-çok değişiyor
- Ama her halükarda bu eğitim size Java'da çok iyi temel verecektir



# Ödev



- <https://www.oracle.com/java/> sayfasını ziyaret edin, linkleri kullanarak gezinin
- Buradaki Download Java linkiyle geline sayfadaki Online Documentation linkini ziyaret edin, Java 25 ile gelen yenilikleri göz atın
- <https://docs.oracle.com/javase/tutorial/> sayfasındaki Java Tutorial'ı makinalarınıza indirerek offline halde ulaşılabilir hale getirin
- <https://dev.java/> sayfasını ziyaret edin, linkleri kullanarak gezinin

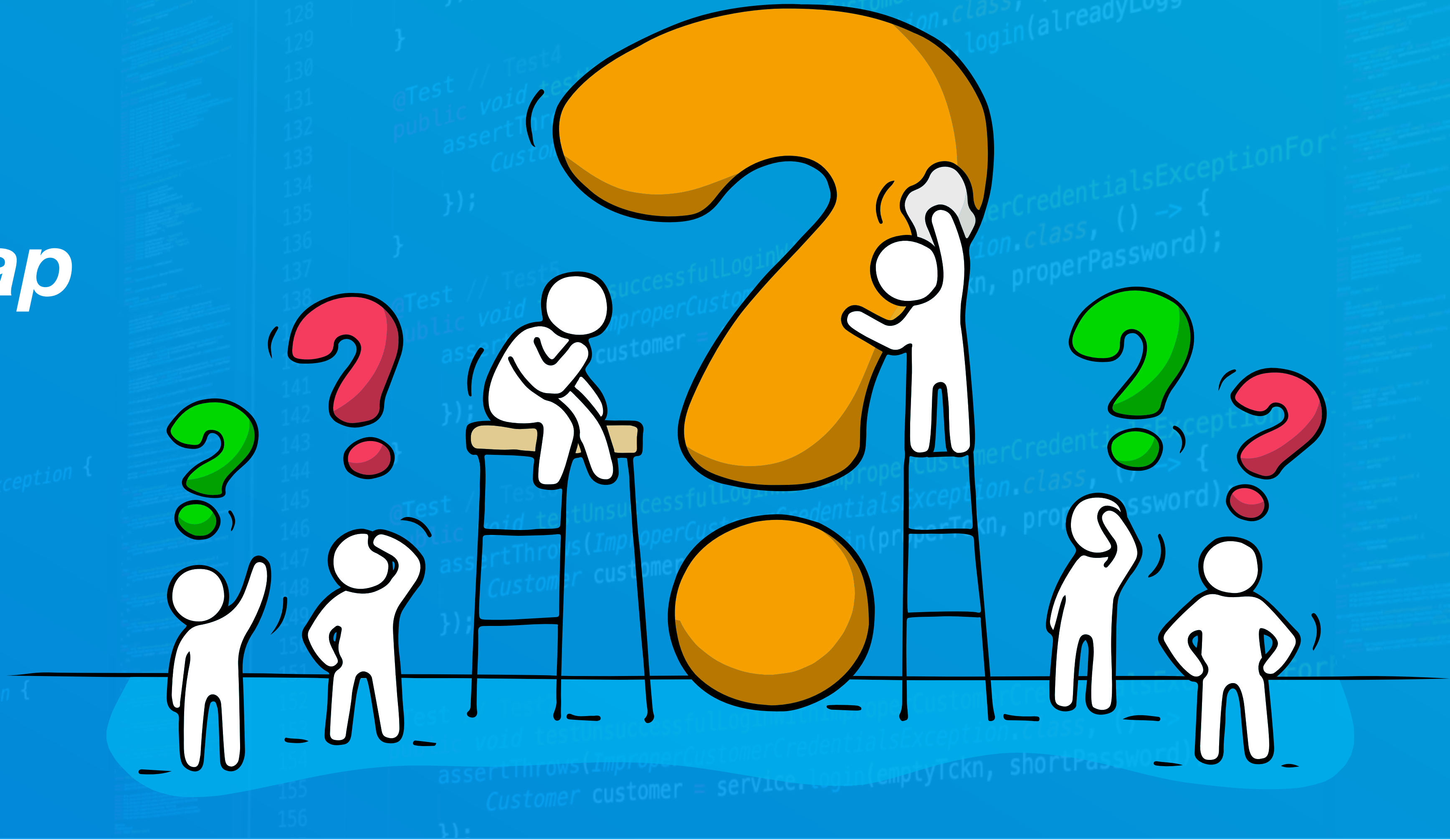


- <https://learn.java/> sayfası ziyaret edin, linkleri kullanarak gezinin
- Burada yazdığımız bazı kodları bu sayfadaki Java Playground'da çalıştırın



- Bir tarayıcınızı Java'ya ya da daha geniş olarak development işlerine özgü kılın, bu tarayıcıda sadece ilgili linkleri açın
- Az önce bahsettiğimiz ve diğer sık kullandığınız Java kaynaklarıyla ilgili linkleri bu tarayıcınızda bookmark yaparak hızlıca ulaşılabilir hale getirin

# Soru ve Cevap Zamanı!



# Neden Java?



# Şu Anda Java



- 1995 yılında büyük beklentiler ile piyasaya çıkan Java, o günden bugüne bu beklentileri karşıladığı rahatlıkla söylenebilir
- Java, çok geniş bir geliştirici topluluğu tarafından kullanılan, zengin, kapalı ve açık kaynak kodlu kütüphanelere sahip, en çok kullanılan yazılım geliştirme dillerinden birisi, muhtemelen en çok kullanılan back-end dilidir
- Dünya çapında 7 ila 10 milyon civarında Java geliştiricisinden bahsedilmektedir

# Java'nın Yaygınlığı



- Java çok geniş bilgi tabanına sahiptir, Amazon'daki kitapların sayısına ya da Stack Overflow araştırmasına <https://survey.stackoverflow.co/2025/> bakılabilir
- Dillerin popülerliği ile ilgili şuralara bakılabilir:
  - TIOBE <https://www.tiobe.com/tiobe-index/>
  - PYPL <https://pypl.github.io/PYPL.html>

# İlk Öğrenilecek Dil Olarak Java



- Java çok paradigmalı (yordamsal, nesne merkezli, fonksiyonel), genel amaçlı bir dil olarak
- hem ilk öğrenilecek dil açısından çok iyi bir alternatiftir
- hem de backend tarafında ilerlemek için en güçlü dildir
- Java'nın söz dizimi (syntax) olarak kurallı olması gözünüzü korkutmasın, öğrendikçe anlayacaksınız ki Java'nın sağlam bir mantığı ve yapısı vardır

# Java Ekosistemi



- Java sadece bir dil değildir, JVM ile birlikte geniş bir ekosistemdir
- Farklı amaçlara yönelik geliştirilmiş pek çok JVM dili vardır
  - Scala, Groovy, Kotlin, Closure, JRuby, Jython, vd

# Java İle Geliştirilen Uygulamalar - I



- Java ile geliştirilmiş web, kurumsal (enterprise), masaüstü (desktop), mobil, gömülü (embedded) vb. tipte pek çok uygulama vardır:
- Amazon, LinkedIn, Netflix, Spotify, vb. pek çok firma
- IntelliJ'in IDEleri, Eclipse, Netbeans IDEleri gibi masaüstü uygulamaları,
- Minecraft gibi oyunlar, JMonkey gibi oyun motorları (game engine),
- Android mobil platformu ve üzerindeki uygulamaların pek çoğu,
- Spring, Hibernate vb. çatılar (framework),

# Java İle Geliştirilen Uygulamalar - II



- Apache vb. pek çok bileşen kütüphanesi (component library)
- Apache'de 400'den fazla proje vardır, en bilinenlerinden bazıları:
  - Kafka
  - Cassandra
  - Hadoop
  - JMeter

# Java'nın Gelişimi - I



- Java'nın gelişimi, Java Community Process (JCP) [www.jcp.org](http://www.jcp.org) tarafından yönlendirilmektedir
- Java, JCP tarafından belirlenen standartlar üzerine kurulmuştur
- Java dili (Java SE), JVM, Java EE ve tüm bileşenler, JCP'de açılan **Java Specification Request (JSR)** ile geliştirilir
- Örneğin 401 nolu JSR <https://jcp.org/en/jsr/detail?id=401>, JavaSE 26 içindir

# Java'nın Gelişimi - II



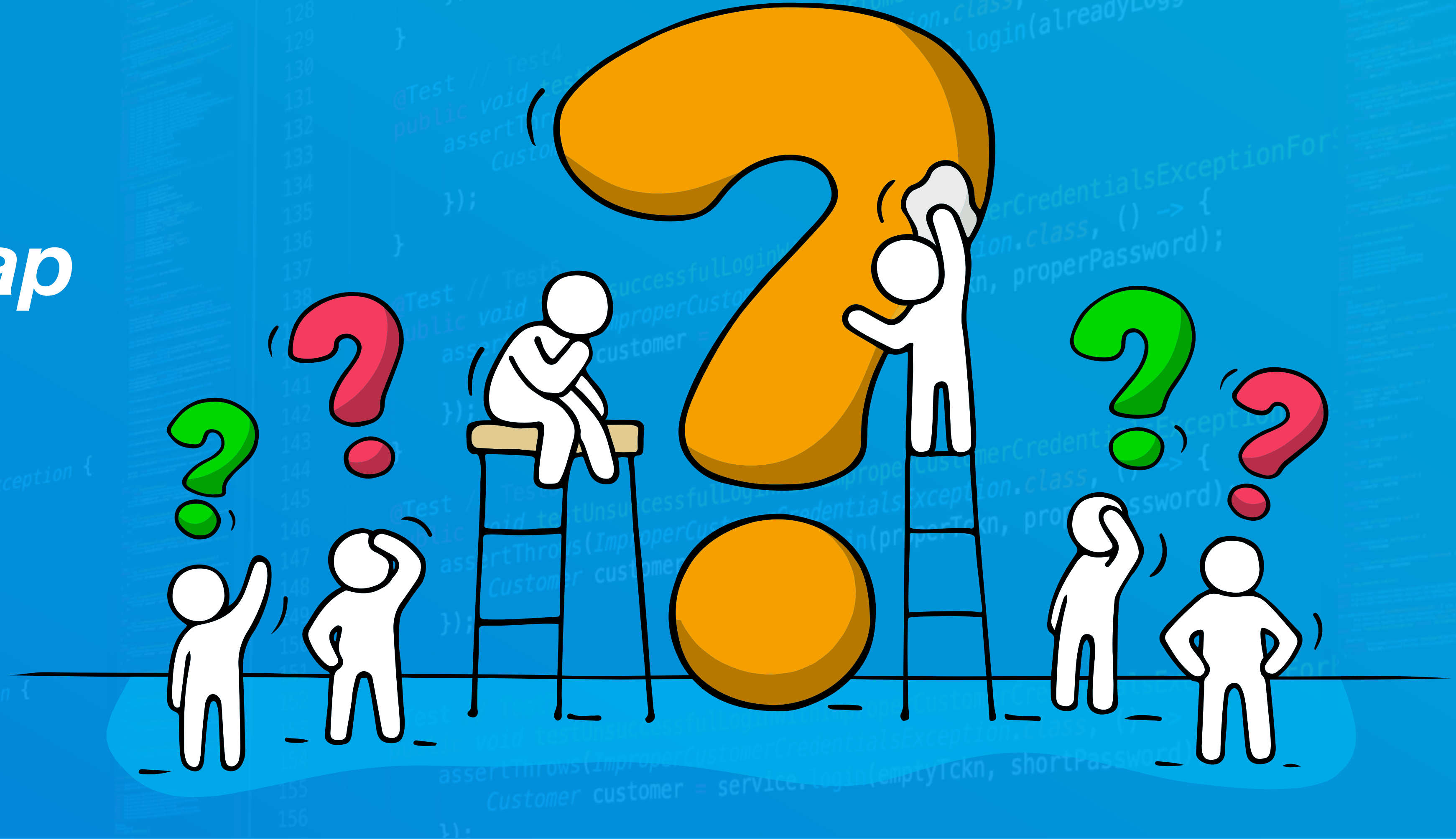
- Her JSR, bileşenin detaylı bir tarifini (specification) yapar ve bu tarifi **referans gerçekleştirme (Reference Implementation, RI)** olarak da kodlar
- JSR ayrıca, yapılabilecek diğer gerçekleştirmeler için uyum testleri (Technology Compatibility Kit, TCK) de hazırlar

# Açık Kaynak Kod



- Java, açık kaynak kod (open source) kültürünün de bir parçasıdır
- Her Java standardının, ticari olan ve olmayanlar yanında açık kaynak kodlu gerçekleştirmeleri vardır
- Java SE'nin kaynak kodu da açıktır: <https://openjdk.org/>

# Soru ve Cevap Zamanı!



# Ödevler



# Ödev - I



- `hw.ch02` paketi altındaki `SumCalculator.java` dosyasındaki geçilen `n` sayısına kadar olan tam sayıların toplamını iki farklı yolla hesaplayan iki metot vardır
- Bu dosyaya bir `main` metot ekleyin ve bu metotta aynı sayıyı bu iki metota geçerek aynı sonucu aldığınızı gözlemleyin

# Ödev - II



- `hw.ch02` paketi altındaki `studentInfo.java` dosyasındaki kodu hatasızca derlenip çalışacak hale getirin
- Dosyanın en başında da yazdığı gibi **package** satırında bir şey değiştirmeyin
- **package** satırının altındaki ve dosyanın sonundaki yorum satırlarını kaldırarak koddaki hataları düzeltmeye başlayın
- IntelliJ size hataları kırmızı renkteki işaretlerle belirtecektir
- AI'dan da yardım alabilirsiniz

# Ödev - III



- Derleme (compilation) ile yorumlama (interpretation) arasındaki temel farklar nelerdir? Derlenen ve yorumlanan dillerde geliştirme sürecinde neler fark eder?
- Java neden hem derlenen hem de yorumlanan bir dildir?
- JDK, JRE ve JVM arasındaki ilişkiyi açıklayın
- Sınıf (class) ile nesne (object) arasındaki ilişki nedir?
- `main` metodun rolü nedir?

# Ödev - IV



- OpenJDK'nın açık kaynak kodlu olduğunu söylemiştik
- Kaynak kodlarına <https://github.com/openjdk/jdk> adresinden ulaşabilirsiniz
- Buradaki `src` linki altında Hotspot JVM dahil olmak üzere tüm JDK'nın kodlarına ulaşabilirsiniz
- Örneğin JVM için `hotspot/os` altında windows, bsd, linux linkleri altındaki C++ kodlarına bakabilirsiniz

# Ödev - V



- Ya da `java.base/share/classes/java/lang` altında `String.java`'ya giderek Java'nın `String` sınıfının kodlarına bakabilirsiniz

# Bundan Sonra?



# Bundan Sonra?



- Bundan sonra Java'nın temellerini ele almaya başlayacağız
- Anahtar kelimelerini, yorumlarını, kod geleneğini ve sabitelerini öğreneceğiz

# Bölüm Sonu

## Soru ve Cevap Zamanı!

